

SnowPro Advanced Data Engineering



MEET THE INSTRUCTOR

Hussain Parbtani

Data Engineering Consultant with a background in Data Engineering, Data Analytics, Data Architecture, Data Security, Cloud Engineering and Cloud Architecture. Focused on cloud-native technologies around data engineering



Twitter
[@hussainparbtani](https://twitter.com/hussainparbtani)

LinkedIn
linkedin.com/in/HussainParbtani



Expertise

- Cloud
- Data Engineering
- Data Analytics
- Snowflake
- AWS
- ETL/ELT

DAY 1

Introduction and Data Movement

Introduction to the Course and Certification

- Overview of SnowPro Advanced: Data Engineer Certification
- Exam format and tips

Snowflake Architecture

- Advanced Features and Capabilities of Snowflake

Data Movement Basics

- Loading diverse datasets into Snowflake
- Understanding and applying various data ingestion techniques

Continuous Data Pipelines

- Design, building, and troubleshooting
- Differentiating various data pipeline types

Connectivity & Data Sharing

- Installing and configuring connectors
- Designing and building data sharing solutions

External Tables in Snowflake

- When and why to use them
- How they work





Performance Optimization

- Diagnosing and optimizing underperforming queries
- Configuring Snowflake for top performance
- Caching features for improved query times

Monitoring and Maintenance

- Tools and best practices for continuous monitoring
- Ensuring health of data pipelines

Storage and Data Protection

- Snowflake's data recovery methods
- Streams and Time Travel
- Analyzing micro-partitions
- Practical applications with Time Travel and cloning

Security in Snowflake

- Core security principles
- System-defined roles and data governance



Advanced Data Transformation

- User-Defined Functions (UDFs)
- Creating and utilizing external functions
- Stored procedures for complex operations

Working with Semi-Structured Data

- Transformation techniques
- Best practices

Introduction to Snowpark

- Leveraging Snowpark for advanced data transformations

Exam Preparation Session

- Mock exams
- Review of critical topics
- Q&A

Course Wrap-up and Feedback

- Key takeaways
- Additional resources and continuous learning
- Course-Survey - Gathering feedback from for course improvement

COURSE OBJECTIVES

Attendees will leave with

1. Understanding advanced techniques for data movement in Snowflake, including ingestion and continuous data pipelines.
2. Knowing how to troubleshoot underperforming queries and optimize performance using caching and other features.
3. Applying storage and data protection principles, such as data recovery and the impact of streams on Time Travel.
4. Managing Snowflake's core security principles and understanding system-defined roles for effective data governance.
5. Learning advanced data transformation methods, including working with UDFs, external functions, and stored procedures.
6. Handling semi-structured data transformations and leveraging Snowpark for complex data operations.
7. Employing Snowflake's best practices in performance optimization, security, and data transformation.
8. Preparing extensively for the SnowPro Advanced: Data Engineer certification, with hands-on practice and mock test sessions

About the Certification

SnowPro Advanced: Data Engineer Overview

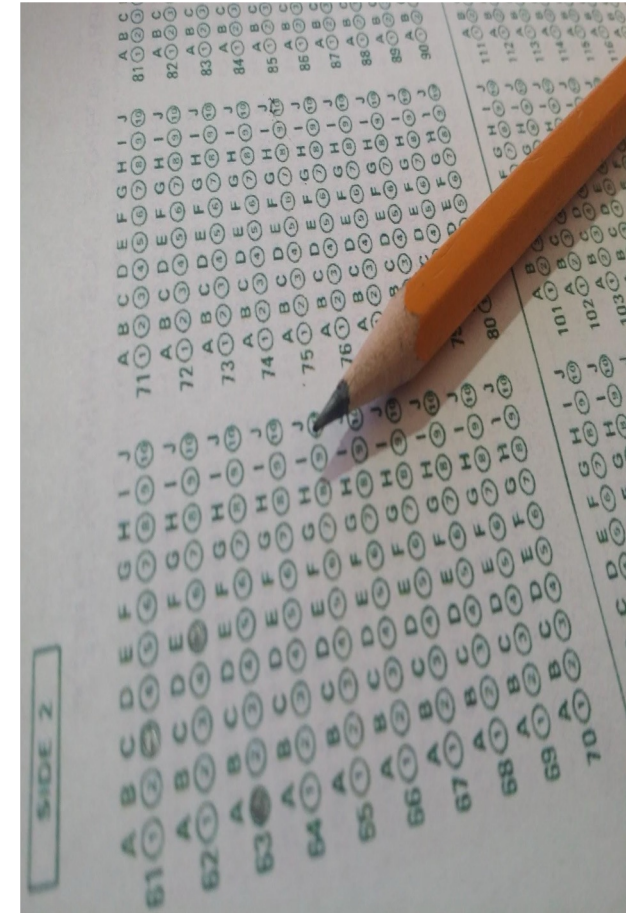
The SnowPro® Advanced: Data Engineer Certification Exam will test advanced knowledge and skills used to apply comprehensive data engineering principles using Snowflake. This certification will test the ability to:

- Source Data From Data Lakes, APIs, and on-premises
- Transform, replicate, and share data across cloud platforms
- Design end-to-end near real-time streams
- Evaluate performance metrics

SnowPro Advanced: Data Engineer Candidate

2+ years of hands-on Snowflake Practitioner experience in a Data Engineering role. Target Audience:

- Data Engineers
- Software Engineers



Exam Breakdown

Total Number of Questions: 65

Question Types: Multiple Select, Multiple Choice

Time Limit: 115 minutes

Language: English

Registration fee: \$375 USD

Passing Score: 750 + Scaled Scoring from 0 - 1000

Unscored Content: Exams may include unscored items to gather statistical information for future use. These items are not identified on the form and do not impact your score, and additional time is factored into account for this content.

Prerequisites: **SnowPro Core Certified**

Delivery Options:

- Online Proctoring
- Onsite Testing Centers

Exam Domain Breakdown

Domain	Domain Weightings on Exams
1.0 Data Movement	25-30%
2.0 Performance Optimization	20-25%
3.0 Storage and Data Protection	10-15%
4.0 Security	10-15%
5.0 Data Transformation	25-30%

INTRODUCTIONS

- Which statement best describes your Snowflake experience?
 - a. I am **currently working** with Snowflake on a project/initiative
 - b. I **expect to work** with Snowflake on a project/initiative in the future
 - c. I am **here to learn** about Snowflake outside of any specific work related project/initiative
- Rate yourself on (1 New Joiner – 5 SME) on
 - Snowflake
 - DBA
 - Data Warehouse
 - SQL
- Expectations for course (please be specific, if possible)
- Why are you here?
- What is your favorite dessert?

LOGISTICS



- **Class Hours:**
- Start time is 8:00am EST
- End time is 4:00pm EST
- Class times may vary slightly for specific classes
- Breaks mid-morning and afternoon (10-15 minutes)



- **Lunch:**
- Lunch is typically noon pm to 1:15pm
- Yes, 1 hour and 15 minutes
- Extra time for email, phone calls, or simply a walk.



- **Telecommunication:**
- Turn off or set electronic devices to vibrate
- Reading or attending to devices can be distracting to other students

- **Miscellaneous**
- Courseware
- Bathroom

Experiment



Content

#00 - Getting Started

Scripts

N/A

Data Loading in Snowflake



Data Loading in Snowflake

Supported File Locations:

- Snowflake uses the term "stage" to describe the location of data files in cloud storage.
- The "COPY INTO <table>" command is used for both:
 - Bulk data loads
 - Continuous data loads via Snowpipe
- This command supports two types of stages:
 - External stages: Cloud storage accounts managed by the user's business entity.
 - Internal stages: Cloud storage within the user's Snowflake account.

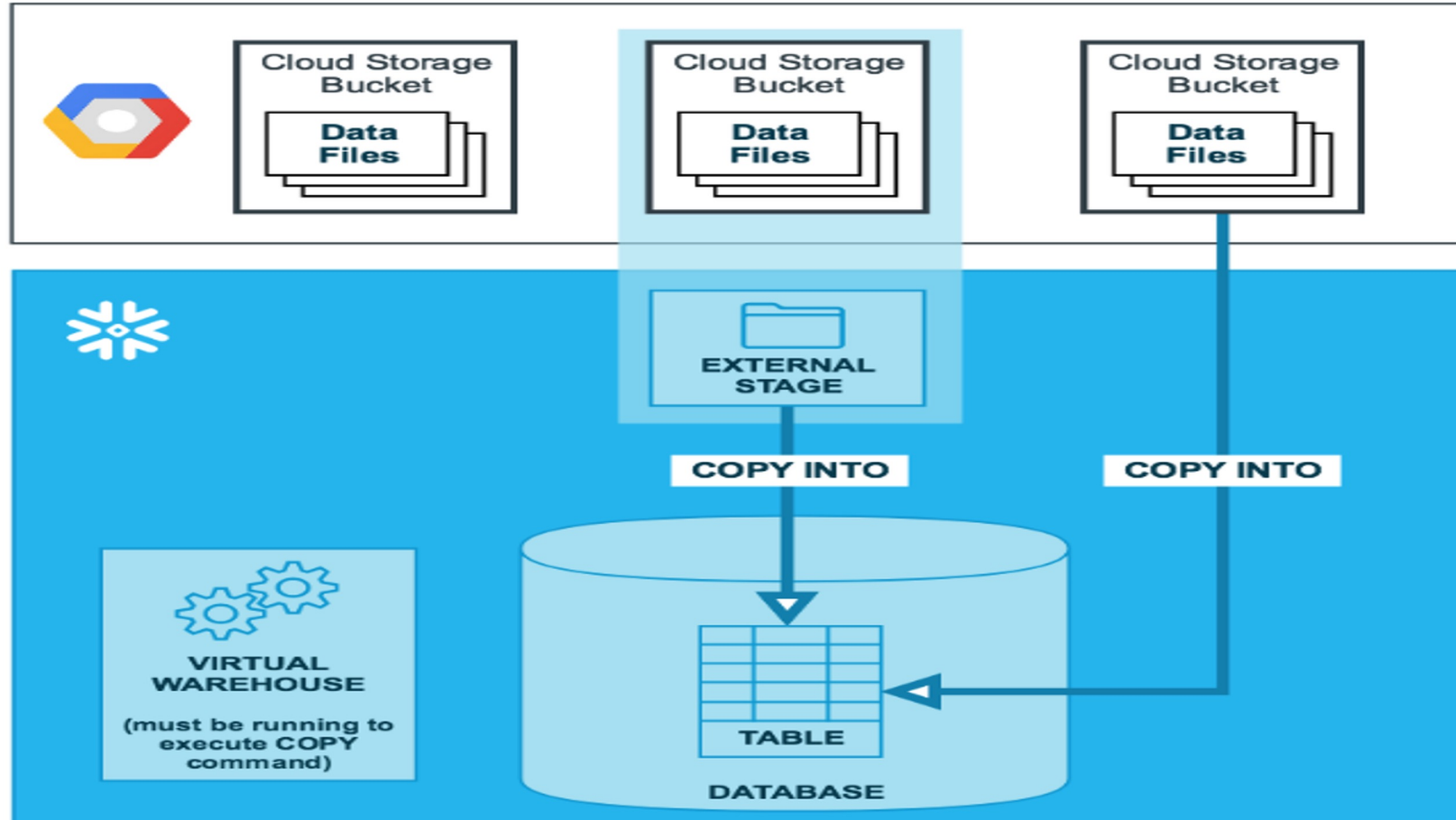
```
COPY INTO mytable  
FROM @my_int_stage;
```

External Stages

- **External stages allow data loading from:**
 - Amazon S3
 - Google Cloud Storage
 - Microsoft Azure
- **Data in archival cloud storage classes, like Amazon S3 Glacier or Azure Archive Storage, is not directly accessible.**
- **Users should upload or stage files using tools from the respective cloud storage service.**
- **"Named external stage" is a database object:**
 - Holds the URL to the staged files.
 - Contains settings for accessing the cloud storage and file format details.
 - Created using the "CREATE STAGE" command.
- **Beware of potential data transfer charges:**
 - May incur if loading data from different regions or platforms than your Snowflake account



External Stages - Example



Internal Stages

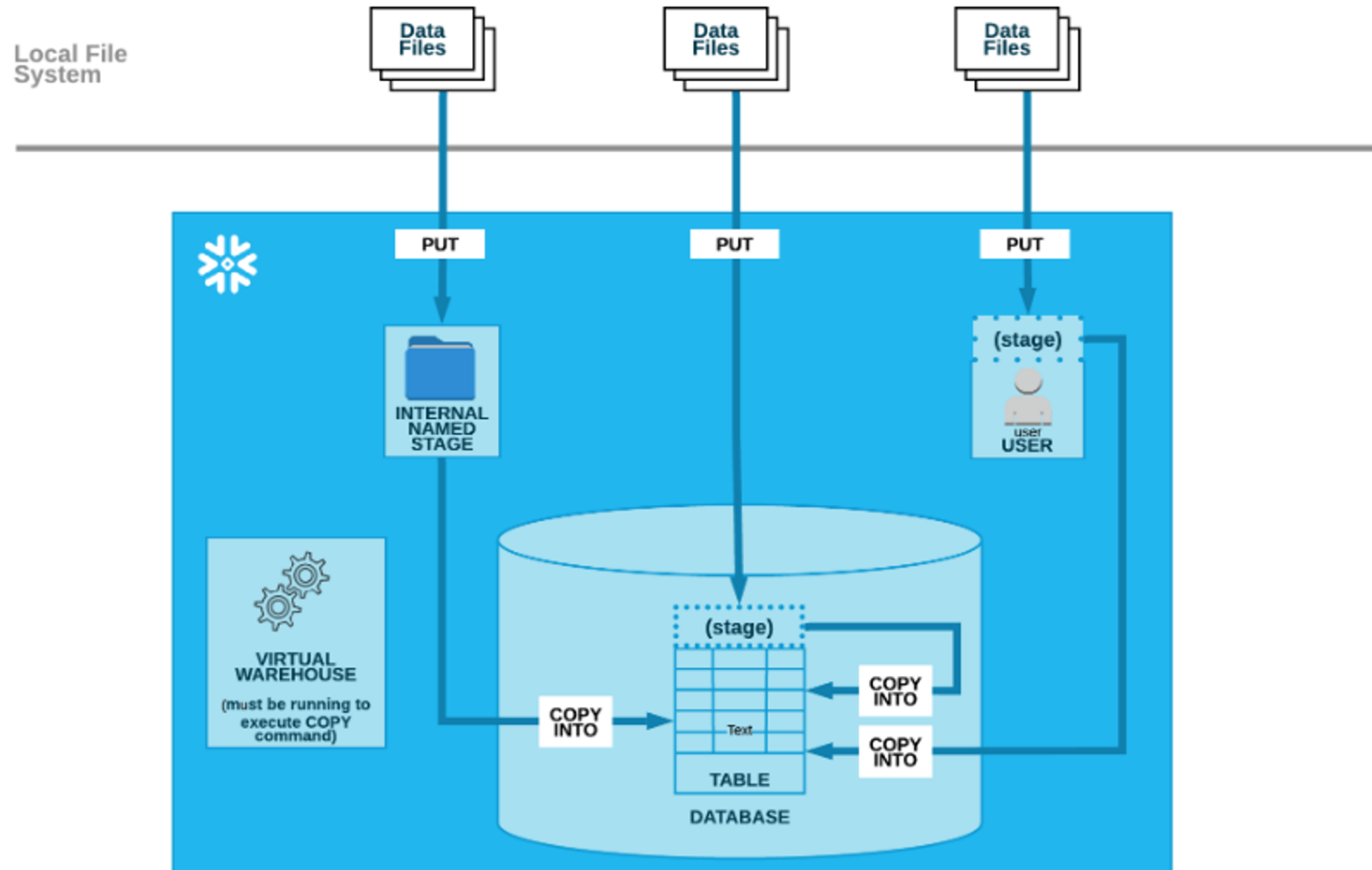
- Internal Stages in Snowflake:

- User Stage:
 - Allocated for each user.
 - Designed for files managed by one user, but can load into multiple tables.
 - Cannot be altered or dropped.
 - Table Stage:
 - Available for each Snowflake table.
 - Stores files for one or multiple users but loads only into its linked table.
 - Implicitly tied to the table; no separate database object.
 - No grantable privileges; table owner required for staging operations.
 - Named Stage:
 - A distinct database object in a schema.
 - Can store files for multiple users and load into various tables.
 - Access and operations controlled by security privileges.
 - Created using the "CREATE STAGE" command.
- Upload to internal stages with the "PUT" command from local systems.

```
PUT file:///C:/data/data.csv @my_stage;
```

 Copy

Internal Stages - Example



Bulk vs Continuous Loading

Bulk Loading with the COPY Command

- Purpose: Load large batches of data from cloud storage or local machines.
- Compute Resources: Utilizes user-provided virtual warehouses.
 - Requires users to size the warehouse based on expected loads.

Continuous Loading with Snowpipe

- Purpose: For smaller volumes of data (micro-batches) with rapid availability.
- Compute Resources: Uses Snowflake's serverless compute model.
 - Auto-scaled and charged per second.
- Features:
 - Loads data minutes after being staged.
 - Provides users with up-to-date results quickly

Bulk vs Continuous Loading - Similarities

- Transformations: Supports data transformations during load, including:
 - i. Column reordering
 - ii. Column omission
 - iii. Casts
 - iv. Truncating long text strings

Load a Subset of Table Data

Load a subset of data into a table. For any missing columns, Snowflake inserts the default values. The following example loads data from columns 1, 2, 6, and 7 of a staged CSV file:

```
copy into home_sales(city, zip, sale_date, price)
  from (select t.$1, t.$2, t.$6, t.$7 from @mystage/sales.csv.gz t)
  FILE_FORMAT = (FORMAT_NAME = mycsvformat);
```

Reorder CSV Columns During a Load

The following example reorders the column data from a staged CSV file before loading it into a table. Additionally, the COPY statement uses the `SUBSTR`, `SUBSTRING` function to remove the first few characters of a string before inserting it:

```
copy into home_sales(city, zip, sale_date, price)
  from (select SUBSTR(t.$2,4), t.$1, t.$5, t.$4 from @mystage t)
  FILE_FORMAT = (FORMAT_NAME = mycsvformat);
```

Continuous Loading in Snowflake

Data Pipelines for Advanced Transformations

- Purpose: Apply complex transformations post data load (ELT vs ETL)
- Workflow:
 - Use Snowpipe to load raw data into a staging table.
 - Transform and optimize data using table streams and tasks.

Snowpipe Streaming API: Directly writes data rows to Snowflake tables without staging.

- Benefits: Lower load latencies and costs.

Snowflake Connector for Kafka:

- Allows for efficient near real-time data streaming.
- Provides a path to benefit from lower latency and cost loads.

Working with Semi-Structured Data Files

Challenges: Semi-structured data can contain thousands of columns.

Solutions:

- Reference directly using external tables.
- Load into VARIANT type column.
- Transform and load into separate columns in a relational table.

Sample JSON Schema

```
{
  "title": "Example Schema",
  "type": "object",
  "properties": {
    "firstName": {
      "type": "string"
    },
    "lastName": {
      "type": "string"
    },
    "age": {
      "description": "Age in years",
      "type": "integer",
      "minimum": 0
    }
  },
  "required": ["firstName", "lastName"]
}
```

Auto-Detecting Schemas

Auto-Detecting Schema

- Purpose: Automatically detect schema of staged semi-structured files and retrieve column definitions.
- Details:
 - Supported Files: Apache Parquet, Apache Avro, ORC, JSON, CSV (Preview for last two).
 - Features: Generate syntax suitable for Snowflake tables, views, and external tables.

Utilizing SQL Functions for Detection

- *INFER_SCHEMA*: Detect column definitions in staged data for Snowflake object creation.
- *GENERATE_COLUMN_DESCRIPTION*: Generate column list using *INFER_SCHEMA* output.
- Note: These functions work for both internal and external stages.

Creating Tables with Detected Definitions

- Procedure:
 - Use `CREATE TABLE ... USING TEMPLATE` or `CREATE EXTERNAL TABLE ... USING TEMPLATE`.
 - Use the *INFER_SCHEMA* function for column detection.
 - Use `COPY` with the `MATCH_BY_COLUMN_NAME` option to load data into structured tables.

Experiment



Content
LAB 2 - Auto
Detect Schemas

Alternatives to Loading Data into Snowflake

External Tables (Data Lake): Query data in cloud storage without loading into Snowflake.

- Source remains in external cloud storage.
- Suitable for large data sets and selective querying.
- Option to create materialized views for better performance.

Working With Amazon S3-compatible Storage

- Capability: Create external stages/tables on software & devices compliant with Amazon S3 API.
- Benefit: Efficient management, governance, and analysis, irrespective of data's physical location.

Considerations of Data Loading

File Sizing Best Practices

- Target file size for optimal load: 100-250 MB compressed.
- Avoid loading files >100 GB.
- For very large files, consider the ON_ERROR copy option.
- Aggregate smaller files; split bigger ones for distributed loading.
- Recommended tools for file splitting:
 - Linux/macOS: split utility.
 - Windows: Third-party tools.



Considerations of Data Loading

Semi-structured data and limitations

- VARIANT data type has a 16 MB size limit per row.
- JSON data: Concatenation of multiple documents.
- Recommended: Use STRIP_OUTER_ARRAY file format option for JSON loading.

Continuous Data Loads & Snowpipe

- Snowpipe loads data within a minute after file notification.
- Overhead costs related to file management in Snowpipe.
- Recommended file size: 100-250 MB for cost-effective Snowpipe loading.
- Use tools like Amazon Kinesis Firehose for data aggregation and batching.

Considerations of Data Loading

Preparing Delimited Text Files

- Default encoding: UTF-8 (others supported).
- Fields with delimiters or carriage returns should be enclosed in quotes.
- Ensure consistent number of columns per row.

Semi-structured Data & Columnarization in Snowflake

- On inserting into VARIANT column, some data extracted to columnar form.
- Elements with “null” values or multiple data types aren't extracted.
- For better performance:
 - Extract “null” elements into relational columns.
 - Use STRIP_NULL_VALUES option.

Considerations of Data Loading

Numeric Data Guidelines

- Avoid commas in numbers (e.g., avoid 123,456).
- Use a decimal point for fractional numbers (e.g., 123456.789).

Date & Timestamp Data Guidelines

- Adhere to supported formats for date, time, and timestamp.
- Mapping considerations for Oracle DATE data type.
- Snowflake checks temporal values at load time; invalid values result in errors.

Considerations of Data Loading

Planning a Load

- Objective: Optimize data load practices for better performance.
- Recommendation: Use separate warehouses for data loading and querying

Staging Data Best Practices:

- Partition data into logical paths incorporating identifiers like geographical location, source, and data write date.
- Organizing by path enables executing concurrent COPY statements on subsets for parallel operations.
- Examples of structured paths:
 - Canada/Ontario/Toronto/2016/07/10/05/
 - United_States/California/Los_Angeles/2016/06/01/11/
- For optimal load performance, narrow the path to the finest level encompassing your data.
- Specific file names or patterns can further confine file selection for loading.
 - Examples:
 - COPY INTO t1 from .../Los_Angeles/2016/06/01/11/mydata;
 - FILES=('mydata1.csv', 'mydata1.csv');
 - PATTERN='.*mydata[^[0-9]{1,3}\$\$].csv';

Considerations of Data Loading

Using Lists of Files for Loading

- COPY INTO <table> supports FILES parameter to load by name.
- Efficient method, but limited to 1,000 files per command.

Pattern Matching for File Selection

- COPY INTO <table> supports PATTERN parameter for regular expression-based selection.
- More flexible, but slower than direct file naming.

Understanding Load Metadata and Loading Older Files

- Snowflake maintains detailed metadata for each table with data load info.
- Load metadata expires after 64 days.
- Files with LAST_MODIFIED date older than 64 days may be skipped.

```
COPY INTO people_data FROM @%people_data/data1/  
PATTERN='.*person_data[^0-9{1,3}$$.csv';
```

Considerations of Data Loading

Handling Files with Expired Metadata

- `LOAD_UNCERTAIN_FILES` Option: Load files with expired metadata without duplicating.
- `FORCE` Option: Loads all files, potentially causing duplication.

Working with JSON and CSV Data

- JSON: Use `STRIP_NULL_VALUES` to avoid storing 'null' as strings.
- CSV: `TRIM_SPACE` option trims leading spaces and considers quotation marks.

Managing Regular Data Loads

Partitioning Staged Data Files

- What: Separate data in Snowflake stages or external locations using logical, granular paths.
- Why: Allows copying data fractionally, optimizing regular imports and ETL processes.
- Examples:
 - s3://bucket_name/application_one/2016/07/01/11/
 - s3://bucket_name/application_two/location_one/2016/07/01/14/
- Note: Consider 10-15 min increment subfolders within hourly folders for optimal performance.

Loading & Removing Staged Data

- Loading:
 - Load files into Snowflake tables by specifying the exact path.
- Removing:
 - Remove loaded staged files to prevent duplication.
 - Caution: Ensure data is loaded successfully before removal. Use COPY_HISTORY to confirm.
 - Methods:
 - Use PURGE copy option during load.
 - Use REMOVE command after successful load.

Note: Efficient removal improves load performance by reducing file scans during COPY commands.

Preparing to Load Data

Supported File Formats

- Structured Data: Delimited formats (e.g., CSV, TSV). Default delimiter: comma.
- Semi-structured Data: JSON, Avro, ORC, Parquet, XML.
 - Auto-detection of compressed files.
 - Load directly into VARIANT columns.
 - Load each top-level object/element as separate rows.
 - Can store in tables with multiple columns if the data is a field in a structured file.

File Formats & Overriding Defaults

- Named File Formats:
 - Encapsulates format information.
 - Streamlines repeated data loads.
 - Can be created via Snowsight, Classic Console, or SQL.
- Overriding Default File Format Options:
 - Set in table, stage definition, or directly during data load.
 - Priority: COPY INTO TABLE statement > Stage definition > Table definition.

Copy Options & Overriding Defaults

- Copy Options: Determines data load behavior such as error handling and data size.
- Overriding Default Copy Options:
 - Set in table, stage definition, or directly during data load.
 - Priority: COPY INTO TABLE statement > Stage definition > Table definition.
 - Note: Options are cumulative, but individual overrides follow the given priority.

Data Unloading in Snowflake

Overview of Data Unloading

- Definition: Exporting data from Snowflake database tables to flat, delimited text files.
- Primary Steps:
 - Use COPY INTO <location> to unload data to a stage.
 - Download file(s) from the stage using either GET (for Snowflake stages) or relevant tools (for Amazon S3 or Microsoft Azure).

Enhanced Unloading Features

- Unloading Using Queries:
 - Use SELECT statement in COPY INTO <location>.
 - Full Snowflake SQL query support, including JOINS.
- File Creation Options:
 - Unload into single or multiple files with SINGLE copy option.
 - Default: SINGLE = FALSE (unload to multiple files).
 - File naming: Prefixes (e.g., "data_") with unique suffixes for parallelization.
 - Control file sizes with MAX_FILE_SIZE option.

Partitioned Data Unloading

- Purpose: Increase efficiency and support varied use cases, like transforming data for data lakes.
- Functionality:
 - Use the PARTITION BY copy option during the unload process.
 - Partition data into a directory structure in cloud storage for optimized consumption by third-party tools.
 - Accepts an expression to partition table rows into separate files.

Snowpipe Overview

What is Snowpipe?

- Snowpipe is a feature of Snowflake designed for continuous data loading. It allows users to automate the data loading process by leveraging event notifications and REST API calls. This is essential for systems with regular, ongoing data input needs, such as streaming applications.

Key Points about Snowpipe:

Data Loading Mechanism: Snowpipe loads data from files as they appear in a stage using the COPY statement.

Pipes: A central component to Snowpipe. They contain the COPY statement which identifies both the source and target of the data loading process.

Loading Mechanisms:

- **Cloud Messaging:** Snowpipe leverages event notifications from cloud storage to inform about the arrival of new data files. This allows for serverless and continuous data loading.
- **REST Endpoints:** Allows client applications to trigger data loads by calling a public REST endpoint with specific details.

Supported Cloud Storage Services: Regardless of where the Snowflake account is hosted (AWS, Google Cloud, or Microsoft Azure), Snowpipe supports a wide range of cloud storage options including Amazon S3, Google Cloud Storage, Microsoft Azure Blob storage, and more.

Snowpipe Overview

Comparison with Bulk Data Loading:

- Authentication: Snowpipe requires JWT signed with RSA encryption for REST endpoint calls.
- Load History: Different storage times for bulk load vs. Snowpipe.
- Transactions: Snowpipe manages transaction sizes based on file content.
- Compute Resources: While bulk loading requires a specified warehouse, Snowpipe uses Snowflake's resources.
- Cost: Bulk loading is billed based on active warehouse time, whereas Snowpipe costs are based on used compute resources.

Recommendations:

- File Size: It's recommended to follow the file sizing guidelines for efficient loading with Snowpipe.
- Load Order: Snowpipe doesn't guarantee the order of data file loading.
- Data Duplication: Snowpipe avoids reloading the same files to prevent data duplication.

Latency: Estimation is complex due to various factors. It's advised to experiment to get an average latency measure.

Security & Access Control:

- Creating Pipes: Specific privileges are required to create and manage pipes.
- Owning Pipes: Owners must have certain privileges over various objects.
- Pausing/Resuming Pipes: Specific roles and privileges are necessary to pause or resume pipes.

Snowpipe DDL: Snowflake offers a set of special DDL commands for managing pipes, such as CREATE PIPE, ALTER PIPE, and more. Additionally, standard access control DDLs can be used for managing permissions.

Snowflake Data Pipelines

What are Data Pipelines?

- Automate transformation and optimization of continuous data loads.
- Efficiently process only new or modified data.

Features of Snowflake's Continuous Data Pipelines:

- Continuous Data Loading:
 - i. Options: Snowpipe, Snowpipe Streaming, Snowflake Connector for Kafka, Third-party tools.
- Continuous Data Transformation:
 - i. Dynamic tables that simplify data engineering.

Change Data Tracking:

- Use of stream objects to record and query delta changes.
- Track when tables are populated and ready for SQL processing.

Recurring Tasks:

- Define recurring schedules for SQL statements.
- Optional integration with table streams for processing new/changed data.
- Supports chaining for complex periodic processing.

Snowflake Streams

What are Streams?

- A Stream in Snowflake is a system-versioned table that tracks changes (inserts, updates, and deletes) to a source table
- Key Point: Streams enable consuming services (such as tasks) to process incremental data changes without any additional data manipulation

Why Use Streams?

- Efficiency - Only the changes (deltas) to tables are processed, reducing the amount of data to be handled.
- Simplicity - Removes the complexity of tracking changes manually.
- Real-time Analytics - Allows for near real-time data processing and analytics.

Components & Concepts

- Base Table: The source table for which changes are being tracked.
- Stream: System-versioned table that captures changes from the base table.
- Consumers: Tasks or processes that read from the stream to handle incremental changes.

Offset Storage in Streams

- **Source Object Initialization:**
 - a. On creation, a stream captures a snapshot of the source object.
 - b. This snapshot marks a specific transactional version, termed the offset.
- **Change Records & Tracking:**
 - a. Post-snapshot, the stream tracks DML changes, showing data states before and after modifications.
 - b. Streams don't store table data but return CDC records through versioning history.
 - c. Hidden columns in the source store change metadata with minimal storage overhead.
- **Stream Utility & Application:**
 - a. Think of streams as bookmarks pointing to specific data moments.
 - b. They're flexible: drop or create them at varied times, even using Snowflake's Time Travel.
 - c. Ideal for data pipelines, ensuring only updated staging data gets transferred.

Table Versioning and Stream in Snowflake

- **Table Version Creation:**
 - a. Every DML transaction commit results in a new table version.
 - b. Applicable to: Standard, Directory, External tables, and underlying tables for views.
- **Stream Offset Dynamics:**
 - a. Stream offset is set between two table versions.
 - b. Stream offers a minimal change set from its offset to the table's latest version.
 - c. Multiple queries can consume this data without altering the offset.
- **Stream Offset Progression:**
 - a. Stream offsets advance solely during DML transactions.
 - b. Autocommit settings impact this: by default, a DML auto-commits, thus potentially advancing the offset.
 - c. Simply querying doesn't move the offset; DML consumption is needed.
- **Tips & Tricks:**
 - a. To push an offset to the current table version without DML consumption: either recreate the stream or use an INSERT with a filtering WHERE clause.
 - b. When a stream is queried within a transaction, the timestamp at the transaction's start is considered.
 - c. Use explicit transaction statements to lock the stream and ensure consistent change record access.

Repeatable Read Isolation in Streams

- **Isolation Overview:**

- a. Streams offer repeatable read isolation.
- b. All SQL statements within a transaction see a consistent set of records in the stream.
- c. This is distinct from table's read committed mode.

- **Delta Record Dynamics:**

- a. Delta records for streams in a transaction span from the stream's present position to the transaction's initiation.
- b. If the transaction commits, stream position progresses to the start time of that transaction.

Repeatable Read Isolation in Streams

Time	Transaction 1	Transaction 2
1	Begin transaction.	
2	Query stream <code>s1</code> on table <code>t1</code> . The stream returns the change data capture records between the current position to the Transaction 1 start time. If the stream is used in a DML statement the stream is then locked to avoid changes by concurrent transactions.	
3	Update rows in table <code>t1</code> .	
4	Query stream <code>s1</code> . Returns the same state of stream when it was used at Time 2 .	
5	Commit transaction. If the stream was consumed in DML statements within the transaction, the stream position advances to the transaction start time.	
6		Begin transaction.
7		Query stream <code>s1</code> . Results include table changes committed by Transaction 1.

Within Transaction 1, all queries to stream `s1` see the same set of records. DML changes to table `t1` are recorded to the stream only when the transaction is committed.

In Transaction 2, queries to the stream see the changes recorded to the table in Transaction 1. Note that if Transaction 2 had begun **before** Transaction 1 was committed, queries to the stream would have returned a snapshot of the stream from the position of the stream to the beginning time of Transaction 2 and would not see any changes committed by Transaction 1.

Overview of Stream Columns and type

Stream Columns

Offset Storage:

- Streams store offsets, not actual table data.
- On querying, it returns historic data in the shape of the source object.

Additional Columns:

- METADATA\$ACTION: Indicates recorded DML operation (INSERT, DELETE).
- METADATA\$ISUPDATE: Represents if operation was part of an UPDATE. Updates in the source object are a pair of DELETE and INSERT in the stream.
- METADATA\$ROW_ID: Provides a unique, immutable ID for tracking row changes.

Types of Streams

Standard:

- Tracks all DML changes: inserts, updates, and deletes.
- Performs join on inserted/deleted rows to provide row-level delta. Rows added and deleted between two transaction points are excluded from delta.
- *Limitation:* Doesn't support geospatial data changes.

Append-only:

- Tracks only row inserts.
- Excludes update and delete operations.
- Beneficial for performance in scenarios like ELT that depend solely on inserts.

Insert-only:

- Exclusive for external tables.
- Monitors only row inserts, ignoring delete operations.
- Cannot access historical records for files in cloud storage. Overwritten/appended files are seen as new, without recording the diff.

Data Retention, Stream Staleness, and Multiple Consumers

Understanding Stream Staleness

Staleness Cause:

- A stream becomes stale when its offset is beyond the data retention period of its source table. When stale, historical data becomes inaccessible.

Avoiding Staleness:

- Consume stream records within the data retention period.
- If unconsumed, Snowflake can extend retention up to 14 days by default to avoid staleness.

Checking Staleness:

- Use DESCRIBE STREAM or SHOW STREAMS commands.
- The STALE_AFTER column shows when a stream might become stale.
- It's critical to consume data before this timestamp.

Recommendations & Best Practices

Stream Recreations:

- When a stream is stale, recreate it using CREATE STREAM.
- If a source table/view is recreated, any stream on it becomes stale.

Cloning & Renaming:

- Cloning a database or schema makes unconsumed records in the cloned stream inaccessible.
- Renaming doesn't break or stale a stream. Dropping and recreating an object with the same name doesn't link old streams to the new object.

Multiple Consumers:

- Each consumer (task, script, etc.) should have its separate stream to avoid different deltas.
- Create multiple streams for the same object for consistent change data.

Key Takeaway: It's crucial to regularly consume streams to avoid staleness, be cautious with operations like cloning, and create separate streams for different consumers.

Streams on Views

Streams on Views Overview

- Streams can be applied to both local and shared views using Snowflake Secure Data Sharing.
- Not applicable for materialized views.
- Underlying tables for the views must be native tables.
- Change tracking should be enabled on the underlying tables.

Supported View Operations

- Projections
- Filters
- Inner or cross joins
- UNION ALL
- Nested views and subqueries in the FROM clause

Unsupported View Operations

- GROUP BY clauses
- QUALIFY clauses
- Subqueries not in the FROM clause
- Correlated subqueries
- LIMIT clauses

Functions in Views

- Functions in the select list must be system-defined, scalar functions.

POP QUIZ:

Snowflake Streams



What is the primary function of a Snowflake stream?

- A) To stream real-time data into Snowflake
- B) To monitor data changes in a table or view
- C) To optimize query performance
- D) To schedule data backups



POP QUIZ:

Snowflake Streams



What is the primary function of a Snowflake stream?

- A) To stream real-time data into Snowflake
- B) To monitor data changes in a table or view
- C) To optimize query performance
- D) To schedule data backups

Explanation: A Snowflake stream is used to monitor changes (inserts, updates, deletes) in a specified table or view. It allows users to capture and process these changes incrementally, rather than having to reprocess the entire table or view.



POP QUIZ:

Snowflake Streams



Which of the following statements are true about Snowflake streams? (Select all that apply)

- A) They can track changes in both tables and views
- B) They require a running Virtual Warehouse to capture changes
- C) They enable time-travel querying
- D) They can be queried like regular tables



POP QUIZ:

Snowflake Streams



Which of the following statements are true about Snowflake streams? (Select all that apply)

A) They can track changes in both tables and views

B) They require a running Virtual Warehouse to capture changes

C) They enable time-travel querying

D) They can be queried like regular tables

A Snowflake stream is used to monitor changes (inserts, updates, deletes) in a specified table or view. It allows users to capture and process these changes incrementally, rather than having to reprocess the entire table or view.



POP QUIZ:

Snowflake Streams



What happens to a Snowflake stream once it is queried (DML)?

- A) It is automatically deleted
- B) It resets and starts monitoring changes from that point forward
- C) It continues to accumulate changes indefinitely
- D) It archives the queried data for future reference



POP QUIZ:

Snowflake Streams



What happens to a Snowflake stream once it is queried (DML)?

- A) It is automatically deleted
- B) It resets and starts monitoring changes from that point forward
- C) It continues to accumulate changes indefinitely
- D) It archives the queried data for future reference

Explanation: When a Snowflake stream is queried (DML), it effectively resets. It starts capturing changes again from that point onward, meaning only the changes that occur after the last query will be captured in the stream.



POP QUIZ:

Snowflake Streams



Which operations can a Snowflake stream capture? (Select all that apply)

- A) INSERT
- B) UPDATE
- C) DELETE
- D) TRUNCATE



POP QUIZ:

Snowflake Streams



Which operations can a Snowflake stream capture? (Select all that apply)

A) INSERT

B) UPDATE

C) DELETE

D) TRUNCATE

Explanation: A standard (i.e. delta) stream tracks all DML changes to the source object, including inserts, updates, and deletes (including table truncates)



POP QUIZ:

Snowflake Streams



What is a requirement for creating a Snowflake stream?

- A) The underlying table must have a primary key
- B) A running Virtual Warehouse
- C) The stream must be linked to a Snowflake task
- D) The underlying table or view must be in the same database



POP QUIZ:

Snowflake Streams



What is a requirement for creating a Snowflake stream?

- A) The underlying table must have a primary key
- B) A running Virtual Warehouse
- C) The stream must be linked to a Snowflake task
- D) The underlying table or view must be in the same database



Experiment



Content
LAB 3 - Streams
on Views

CHANGES Clause: Read only Alternative to Streams

CHANGES Clause Overview

- A read-only alternative to streams in Snowflake.
- Queries change tracking metadata for tables/views.
- Allows querying between two time points without creating a stream.
- No advancement in offset: Multiple queries can retrieve changes between different transactional points.
- Requires specifying a start point with an AT | BEFORE clause; optional end point with END clause.

Streams vs. CHANGES Clause

- Streams store the current transactional table version - ideal for most Change Data Capture (CDC) scenarios.
- CHANGES clause is for infrequent use cases where managing the offset for arbitrary time periods is needed.

Requirements for Change Tracking Metadata

- For Tables:
 - Enable change tracking (using ALTER TABLE ... CHANGE_TRACKING = TRUE) or
 - Create a stream on the table (using CREATE STREAM).
- For Views:
 - Enable change tracking on the view and its underlying tables.

Experiment



Content
LAB 4 - Changes
Clause

Billing for Streams

Streams and Data Retention

- When a stream isn't consumed regularly, Snowflake extends the data retention period of its source table/view.
- If data retention is less than 14 days:
 - Snowflake extends it to the smaller of stream's transactional offset or 14 days.
 - This extension applies irrespective of the Snowflake edition you're using.

Implications on Billing

- Extended data retention requires more storage.
- This increase will be noticeable in your monthly storage charges.

Cost Elements of Streams

Storage Cost - Due to extended data retention

Processing Cost - Incurred when a virtual warehouse queries the stream.

- This will reflect as Snowflake credits on your bill.

Snowflake Tasks

Task Overview:

- Tasks execute:
 - Single SQL statements
 - Stored procedure calls
 - Procedural logic with Snowflake Scripting
- Can work with table streams for continuous ELT workflows.
- Useful for generating periodic reports or other regular operations

Use Cases

With Table Streams: Ensure "exactly once" semantics for table changes.

Standalone: Generate reports, insert/merge rows, or other periodic operations.

Compute Resources for Tasks

- Two compute models:
 - Snowflake-managed (Serverless)
 - User-managed (Virtual Warehouse)

Serverless Tasks

Serverless Tasks

- Rely on Snowflake-managed compute resources.
- Resources auto-scale based on workload.
- Ideal size is determined by analyzing statistics from previous runs.
- Maximum size equals XXLARGE warehouse.
- Shared compute resources among various workloads in an account.

Creating Serverless Tasks

- To create, omit the WAREHOUSE parameter.
- The executing role must have EXECUTE MANAGED TASK privilege.
- Syntax is almost the same as for user-managed tasks.

Billing & Limitations

- Billing for serverless tasks differs from standard credit consumption model.
- Limitations:
 - Can't invoke UDFs with Java/Python code.
 - Can't call stored procedures in Scala or those that call Java/Python UDFs.

User-Managed Tasks

User Managed Tasks Overview

- Tasks can utilize your existing virtual warehouses.
- You're responsible for selecting an appropriately sized warehouse for the task's SQL actions.

How to Choose a Warehouse Size?

Analyze Average Run Time:

- Use the TASK_HISTORY Account Usage view.
- Measure the difference between scheduled and completed times.

Optimize SQL Statements:

- Rewriting or using stored procedures.

Factor in Queuing Time:

- Tasks might be queued if the warehouse resources are busy.

Deciding Between Serverless and User Managed Tasks

Category	Serverless Tasks	User-managed Tasks
Concurrency and Duration	Good for fewer tasks or quick tasks (<1 min). Stable runs suit serverless.	Better for fully utilized warehouses with many concurrent tasks. Suited for unpredictable workloads.
Billing	Based on Actual Usage	Based on Warehouse Size, with 60-sec minimum
Scheduling Adherence	Ideal when strict scheduling is crucial. Auto-scaled resources if tasks run close to their entire interval	Suitable when scheduling is Flexible

Important Notes

- Maximum size for serverless tasks is **XXLARGE** warehouse.
- If more resources are needed, use user-managed tasks with the desired warehouse size.

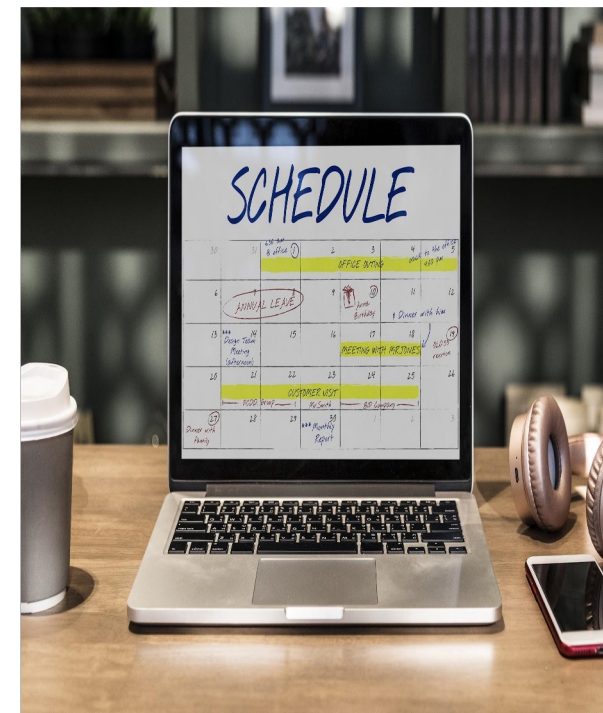
Task Scheduling in Snowflake

Regular Scheduling:

- Standalone tasks and root tasks in a DAG run on schedules.
- If a task overlaps its next scheduled time, it's skipped.
- Warehouse size should match task load.

Daylight Saving Time (DST) Concerns:

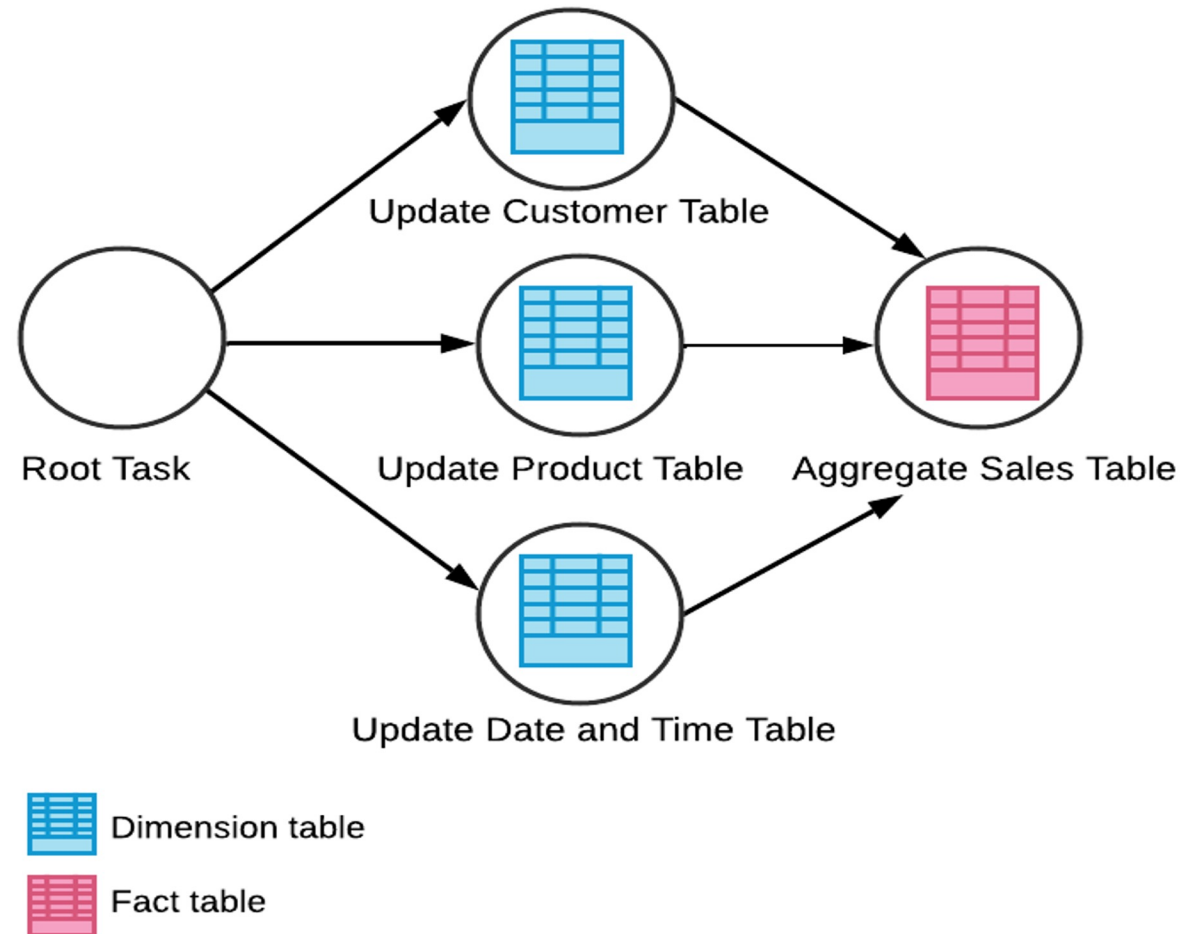
- Tasks can run twice during the fall DST change or skip in spring.
- Solutions:
 - Avoid scheduling between 1 AM - 3 AM on DST days.
 - Manually adjust cron expressions.
 - Use UTC, which doesn't consider DST.



Directed Acyclic Graphs & Task Ownership

- **DAG Overview:**
 - Comprises a root task and additional tasks defined by dependencies.
 - Task runs only after its predecessors complete successfully.
 - DAGs limited to 1000 tasks. A single task can have 100 predecessors and 100 child tasks.
- **Task Ownership:**
 - All tasks in a DAG must have the same owner and be in the same database and schema.
 - Transferring ownership of a task breaks its dependencies.
 - If ownership of all DAG tasks is transferred, relationships remain intact.

Snowflake Tasks - DAG



Overlapping DAG Runs in Snowflake

Default Behavior:

- One DAG instance runs at a time.
- If DAG run exceeds schedule, next run might be skipped.

ALLOW_OVERLAPPING_EXECUTION Parameter:

- Default: FALSE.
- If TRUE, allows overlapping of DAG runs.

Handling Overlaps:

- Important if overlapping runs produce incorrect/duplicate data.
- Align root task schedule and warehouse size, or consider Snowflake-managed resources.
- Consider:
 - Increasing scheduling time.
 - Modifying compute-heavy tasks.
 - Parallel processing in SQL.
 - Setting ALLOW_OVERLAPPING_EXECUTION to TRUE if necessary.

Managing and Viewing DAGS in Snowflake

Viewing Dependent Tasks:

- Query the TASK_DEPENDENTS table function.

DAG Runs with Suspended Child Tasks:

- Child tasks can be suspended independently.
- If root is suspended, child tasks can still be resumed.
- A run will ignore suspended child tasks.

Resuming Suspended Tasks:

Use SYSTEM\$TASK_DEPENDENTS_ENABLE to recursively resume all tasks.

Severing Dependencies:

- Dependencies can be broken by:
 - Removing predecessors (ALTER TASK ... REMOVE AFTER).
 - Dropping predecessors (DROP TASK).
 - Transferring task ownership (GRANT OWNERSHIP).
- Severed child tasks become standalone or root tasks, depending on existing dependencies.

Snowflake Tasks

Versioning Overview:

- Version set when a task is resumed or manually executed.
- If a root task, entire DAG's version is set.

Modification Conditions:

- To modify a task in a DAG, root task must be suspended.
- Current executing tasks use the current version.
- New versions are set when tasks are resumed.

Stored Procedure Definition Changes:

- Can affect ongoing DAG if called during execution.

Version History:

- Query TASK_VERSIONS Account Usage view in the SNOWFLAKE shared database.

Session Parameters:

- Session parameters can be set for task sessions.
- Modify with ALTER TASK.
- Note: Tasks do not support account or user parameters.

Automatically Suspending Failed Tasks:

- Can suspend tasks after consecutive failed runs.
- Parameter: SUSPEND_TASK_AFTER_NUM_FAILURES.
- Behavior:
 - Standalone tasks: suspend after consecutive failures.
 - Root tasks in a DAG: suspend after any single task fails consecutively.

Task Execution

Task Execution:

- Manual Task Execution:
 - Use the EXECUTE TASK command.
 - Useful for testing tasks or integrating with external data pipelines.
- Viewing Task History:
 - For individual task history: Query TASK_HISTORY table function.
 - For current DAG runs: Query CURRENT_TASK_GRAPHs table function.
 - For DAG runs in the past 60 minutes:
 - Query COMPLETE_TASK_GRAPHs table function or view.

Task Costs:

- User-managed warehouse: You are billed based on the warehouse usage while the task is running.
- Snowflake-managed resources (serverless compute model): You're billed for actual compute usage. A 1.5x multiplier is applied to manage the costs. Nevertheless, this might be more cost-effective than user-managed warehouses.
- You can check how many credits are used by referring to the “Serverless Feature Credit Table”. The SERVERLESS_TASK_HISTORY table function or view can be used to retrieve credit usage.

Understanding the System Service:

- Tasks run with the task owner's privileges but aren't associated with a user.
- The system service (SYSTEM) executes these tasks and isn't tied to a user in the account, ensuring security and smooth operations.

Snowflake Tasks - Continued

Task DDL:

- Snowflake provides Data Definition Language (DDL) commands specifically for tasks like CREATE TASK, ALTER TASK, DROP TASK, etc.

Task Functions:

- Functions to retrieve task-related info, like SYSTEM\$CURRENT_USER_TASK_NAME, TASK_HISTORY, and TASK_DEPENDENTS.

Task Security:

- Creating Tasks: Requires certain privileges on the Account, Database, Schema, and Warehouse.
- Owning Tasks: The owner of a task (the role with OWNERSHIP privilege) must have specific privileges.
- Resuming or Suspending Tasks: Apart from the owner, roles with the OPERATE privilege on the task can also suspend or resume tasks.
- Creating a Task Administrator Role: It's suggested to make a custom role (e.g., taskadmin) and assign certain privileges to it for ease.

Dropping a Task Owner Role:

- If a task owner role is deleted, the task is "re-posessed" by the role that dropped the owner role, ensuring a clear chain of custody and responsibility.
- A running task that's under a role will complete its processing even if the role is dropped.

Workflow:

- To set up a task:
Complete the steps in "Creating a Task Administrator Role".
Create the task using CREATE TASK. By default, the task will be suspended.
To activate the task, use ALTER TASK ... RESUME.

POP QUIZ:

Snowflake Tasks



What is the primary purpose of Snowflake Tasks?

- A) To automatically update data models
- B) To schedule and execute SQL statements
- C) To generate reports for business analytics
- D) To optimize database storage



POP QUIZ:

Snowflake Tasks



What is the primary purpose of Snowflake Tasks?

- A) To automatically update data models
- B) To schedule and execute SQL statements
- C) To generate reports for business analytics
- D) To optimize database storage

Explanation: Snowflake tasks are designed for scheduling and executing SQL statements at specific times or intervals. They automate repetitive SQL operations, such as data transformation, aggregation, or other routine database maintenance activities.



POP QUIZ:

Snowflake Tasks



Which of the following are features of Snowflake tasks? (Select all that apply)

- A) Support for cron-like scheduling
- B) Real-time data streaming
- C) Ability to chain tasks together
- D) Automatic scaling of compute resources



POP QUIZ:

Snowflake Tasks



Which of the following are features of Snowflake tasks?
(Select all that apply)

A) Support for cron-like scheduling

B) Real-time data streaming

C) Ability to chain tasks together

D) Automatic scaling of compute resources

Explanation:

- A) Snowflake tasks support cron-like scheduling, allowing users to specify execution times using a cron expression.
- B) Real-time data streaming is not a direct feature of tasks; it's more related to data ingestion mechanisms.
- C) Tasks can be chained together, meaning the completion of one task can trigger the execution of another.
- D) Automatic scaling of compute resources is a feature of Snowflake's warehouses, not directly related to tasks.

POP QUIZ:

Snowflake Tasks



How can tasks in Snowflake be triggered? (Select all that apply)

- A) At a specific time using a schedule
- B) By an external event or API call
- C) By the completion of another task
- D) Manually by a database administrator



POP QUIZ:

Snowflake Tasks



How can tasks in Snowflake be triggered? (Select all that apply)

A) At a specific time using a schedule

B) By an external event or API call

C) By the completion of another task

D) Manually by a database administrator

Explanation:

- A) Tasks can be scheduled to run at specific times using cron-like scheduling expressions.
- B) Tasks are not typically triggered by external events or API calls; they rely on schedules or dependencies within Snowflake.
- C) One task can trigger another, creating a chain of tasks where the completion of one leads to the start of the next.
- D) Tasks are not typically triggered manually by administrators; they are designed for automation.

POP QUIZ:

Snowflake Tasks



In Snowflake, what can be a consequence of improperly managed task schedules?

- A) Reduced data security
- B) Overutilization of compute resources
- C) Automatic deletion of data
- D) Increased storage costs



POP QUIZ:

Snowflake Tasks



In Snowflake, what can be a consequence of improperly managed task schedules?

- A) Reduced data security
- B) Overutilization of compute resources
- C) Automatic deletion of data
- D) Increased storage costs

Explanation: Improperly managed task schedules can lead to overutilization of compute resources. For instance, if tasks are scheduled too frequently or overlap significantly, they can consume excessive compute power, potentially leading to higher costs and reduced efficiency. It doesn't directly affect data security, lead to automatic data deletion, or increase storage costs.



POP QUIZ:

Snowflake Tasks



What is a prerequisite for a Snowflake task to execute?

- A) A running Virtual Warehouse
- B) An active Internet connection
- C) A predefined user role
- D) A stored procedure



POP QUIZ:

Snowflake Tasks



What is a prerequisite for a Snowflake task to execute?

A) A running Virtual Warehouse

B) An active Internet connection

C) A predefined user role

D) A stored procedure

Explanation: For a Snowflake task to execute its SQL statements, it requires a running Virtual Warehouse (compute resources) to perform the computations. Without an active warehouse, the task will not be able to execute its operations.



Dynamic Tables in Snowflake

What are Dynamic Tables?

- Building blocks for declarative data transformation pipelines.
- Automate data transformation without managing pipelines.
- Table's content is determined by a specified query, refreshed regularly.

Benefits of Dynamic Tables:

- No need for streams or tasks for refreshes.
- Ideal when code management and data dependencies aren't desired.
- Best for materializing results of multi-table queries.
- Simplified ETL pipelines with target data freshness.

Creation & Functionality:

- Specify the SQL query for desired results.
- Automated refreshes keep data up to date based on target freshness.

Chaining & Dependencies:

- Can query other dynamic tables.
- Refresh ensures all dependent tables are up-to-date.

Privileges & Features:

- Specific privileges for managing dynamic tables (SELECT, OPERATE, OWNERSHIP).
- Supports Snowflake Time Travel for accessing historical data.
- Replication for disaster recovery or data sharing.

Understanding Dynamic Table Refresh

Dynamic Table Refresh in Snowflake:

- Dynamic tables reflect content based on specific query results. They update (or refresh) when underlying data changes.
- When created, a dynamic table undergoes a full refresh to ensure snapshot isolation.
- You can track the progress of this initial refresh using `information_schema.dynamic_table_refresh_history()`.

Refresh Types:

Incremental refresh: Analyzes the query for the dynamic table and calculates changes since the last refresh. This refresh type only merges the changes.

Full refresh: If incremental refresh isn't possible, a full refresh happens. This refresh type involves re-executing the query and replacing the previously materialized results.

Target Lag:

- Refers to the delay or gap between updates in the base tables and the dynamic table's content.
- Can be defined either as a measure of freshness (e.g., `TARGET_LAG = 10 minutes`) or as `DOWNSTREAM`, which means it refreshes when other dependent dynamic tables need to refresh.

Understanding Dynamic Table Refresh

Supported Queries for Incremental Refresh:

- Dynamic tables support a range of queries for incremental refresh, including:
 - Common table expressions with the WITH clause.
 - Deterministic built-in functions and immutable user-defined functions in the SELECT clause.
 - Source tables, views, and other dynamic tables in the FROM clause.
 - All window functions with the OVER clause.
 - Filters in the WHERE clause.
 - Inner joins, outer joins, and cross joins.
 - UNION ALL.
 - GROUP BY.

However, some constructs and queries do not support incremental refreshes. For such cases, a full refresh is utilized, potentially leading to higher costs

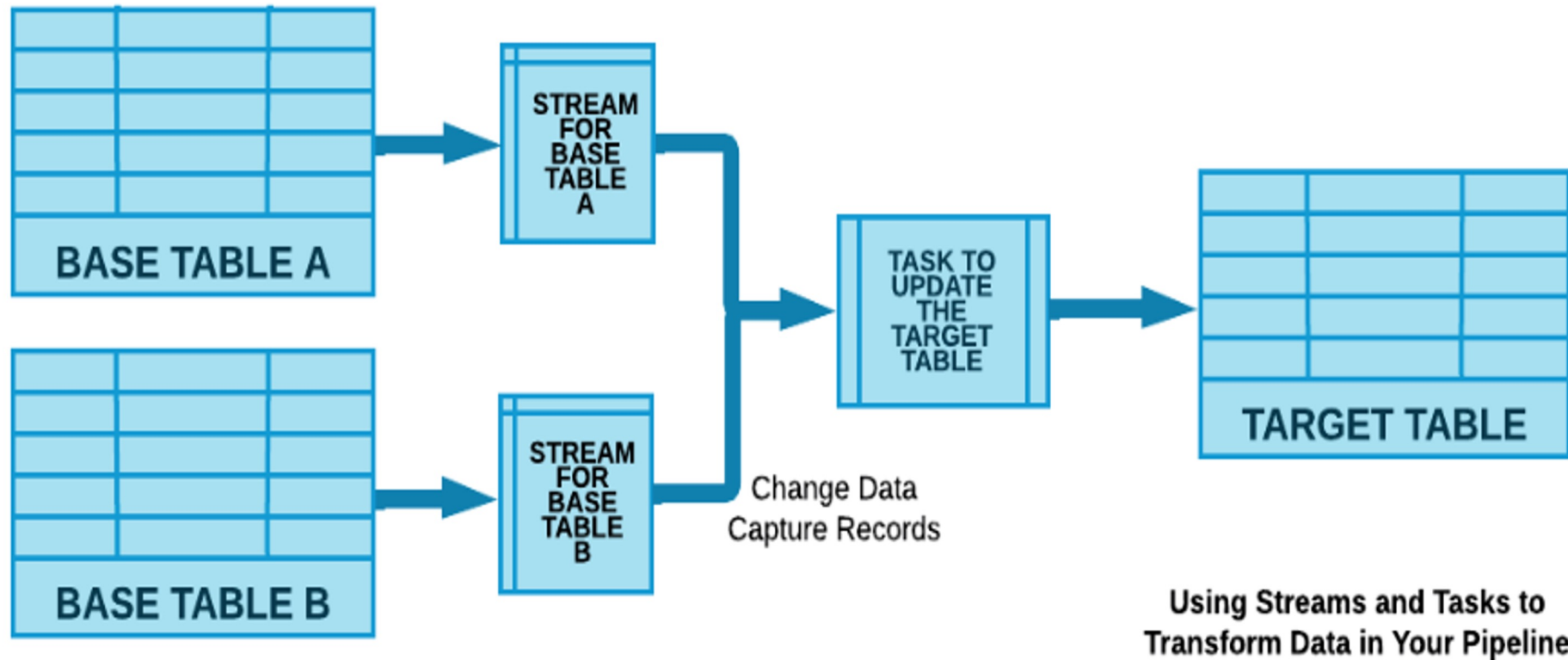
Refresh Schedule with Interdependent Dynamic Tables:

- If a dynamic table depends on another, Snowflake ensures both tables refresh at compatible times.
- Target lag is an aim, not a guarantee. Factors like warehouse size, data size, and query complexity can influence actual refresh times.
- Snapshot isolation ensures that you query the same data "snapshot" across dependent dynamic tables at any given time.

Overall, understanding the dynamic table refresh process in Snowflake is crucial for users who want to ensure timely data availability and consistency across their dynamic tables. The incremental and full refresh modes, combined with the concept of target lag, offer flexibility and efficiency in how data updates in these tables.

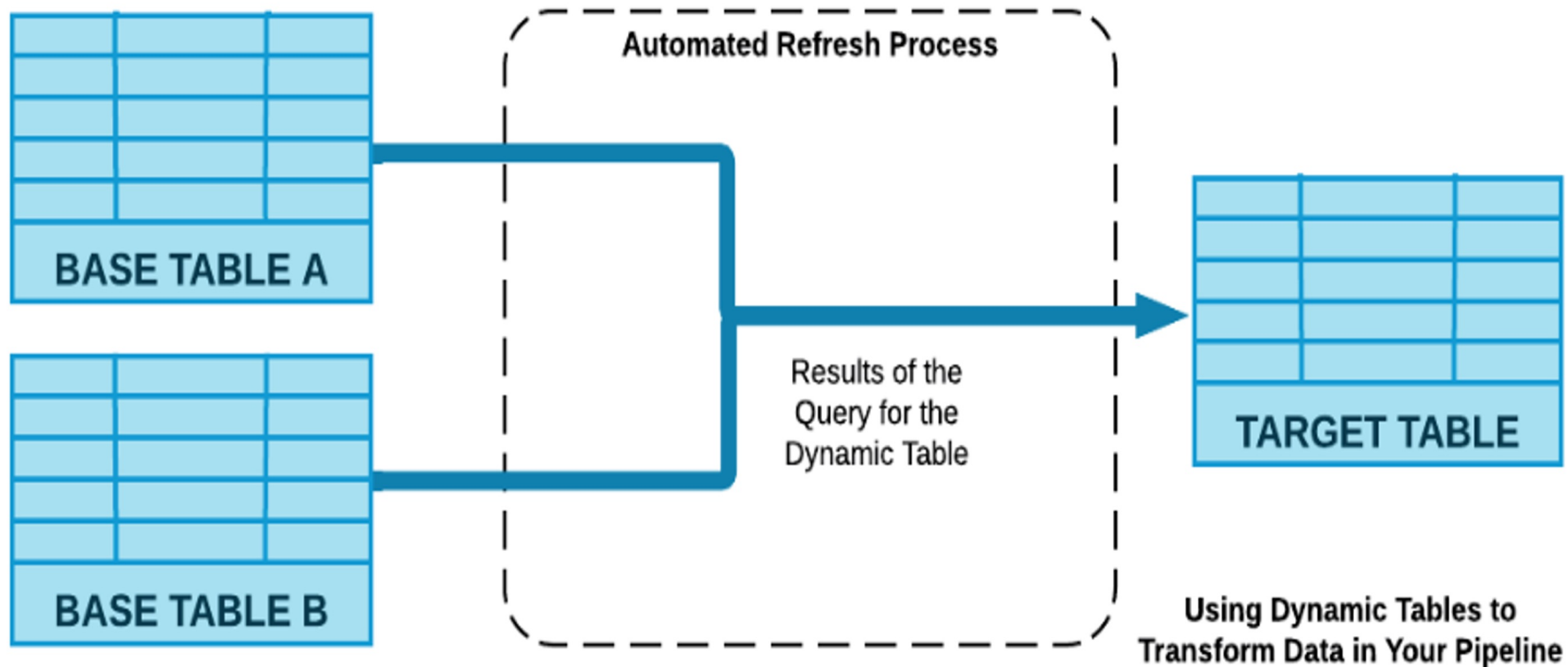
Dynamic Tables Compared to Streams and Tasks

Streams and Tasks in Snowflake



Dynamic Tables Compared to Streams and Tasks

Dynamic Tables in Snowflake:



Dynamic Tables Compared to Streams and Tasks

Streams and Tasks

Programming Approach:

Imperative approach, where procedural code is written to dictate how data should be transformed.

Scheduling:

User defines a schedule for execution.

Functionality:

Can call non-deterministic code, stored procedures, UDFs, external functions, and other tasks.

Refresh Mechanism:

Can use streams for incremental data refreshes. Tasks are scheduled to run periodically.

Dynamic Tables in Snowflake

Programming Approach:

Declarative approach, where a SQL query specifies the desired result, and the system handles data retrieval and transformation.

Scheduling:

An automated process determines refresh schedules based on desired data "freshness" or lag.

Functionality:

Limited to SQL joins, aggregations, window functions, etc. Cannot call stored procedures, tasks, or certain types of queries.

Refresh Mechanism:

An automated process performs regular incremental refreshes based on the target freshness specified.

Dynamic Tables Compared to Materialized Views

Materialized Views

Primary Use Case:

Aimed at transparently enhancing query performance. The system can rewrite queries to use materialized views instead of base tables.

Query Complexity:

Can only use a single base table and cannot be based on complex queries.

Data Currentness:

Always current. Any changes to the base table will update the view or use the updated data.

Dynamic Tables in Snowflake

Primary Use Case:

Designed for multi-level data pipelines. Query optimizer doesn't rewrite queries to use dynamic tables unless specified in the query.

Query Complexity:

Can be based on complex queries, including those with joins and unions.

Data Currentness:

Data is current up to the specified target lag time.

Understanding the Costs of Dynamic Tables

1. Storage:

- Like any other table in a database, dynamic tables occupy space, hence incurring storage costs.

2. Cloud Services Compute:

- Dynamic tables require cloud services to check for changes in the underlying base objects and trigger necessary refreshes.
- Pricing: Snowflake has an interesting model here: it only bills you for cloud services compute if the daily cost for these services is greater than 10% of the daily warehouse cost for the account. This could provide some cost relief in scenarios where the cloud service operations are minor compared to overall warehouse usage.

3. Virtual Warehouse Compute:

- A virtual warehouse is essentially the compute infrastructure that runs your queries. Dynamic tables need this for both initial data population and subsequent refreshes based on changes in base data.



Experiment



Content
LAB 5 - Dynamic
Tables

User Defined Functions

- **What are UDFs?**
 - UDFs are custom functions created by users to perform operations that might not be achievable using the system's built-in functions
 - Enable tailored data manipulation and transformation
 - Extend SQL's native capabilities
- **Why use UDFs?**
 - Address specific data transformation challenges
 - Encapsulate complex logic for reusability
 - Achieve operations unique to business or data needs

Types of UDFs in Snowflake

- **Types of UDFs in Snowflake?**
 - Scalar UDFs: Return a single value based on input values
 - Table UDFs: Returns a table of values
- **Benefits and Considerations for UDFs**
 - Benefits: Code reusability, tailed transformations, enhanced Functionality
 - Consideration: Potential for performance overhead, need for thorough testing, ensuring UDFs are well-documented

Scalar User Defined Functions

- Accept one or more parameters
- Return a single value
- Often used for calculations, data transformation, and string manipulations.
- Can be defined using SQL, Java, Javascript, Python and Scala

Example:

```
CREATE FUNCTION fahrenheit_to_celsius(f FLOAT)
RETURNS FLOAT
AS 'return (f-32)*5/9;';
```

Table User Defined Functions

- Accept parameters and returns a table (multiple rows of data)
- Suitable for transformation that result in multiple rows or filtering data
- Can be defined using SQL, Java, Javascript, and Python (not Scala)

Example:

```
CREATE FUNCTION custom_sales_view(start_date DATE, end_date DATE)
RETURNS TABLE (sale_date DATE, total_amount FLOAT)
AS 'SELECT sale_date, total_amount FROM sales WHERE sale_date BETWEEN
start_date AND end_date;';
```


Best Practices for UDFs

- Maintain simplicity where possible
- Test all UDFs in a non-production environment first
- Document the purpose, inputs, and outputs, and usage examples.
- Monitor performance, especially with non-sql UDFs

UDFs vs Stored Procedures

- **Stored Procedure Purpose**

- Generally to perform administrative operations by executing SQL statements. The body of a stored procedure is allowed, but not required, to explicitly return a value (such as an error indicator)

- **User Defined Function Purpose**

- Calculate and return a value. A function always returns a value explicitly by specifying an expression. For example, the body of a JavaScript UDF must have a return statement that returns a value.

When to create a UDF or a Stored Procedure

- **Create a stored Procedure when**

- You're migrating an existing stored procedure from another application/system.
- You need to perform DDL or DML database operations:
 - Administrative tasks, including DDL such as deleting temporary tables, deleting data older than N days, or adding users.
 - DML statements (UPDATE statements, for example)

- **Create a UDF when**

- You're migrating an existing UDF from another application/system.
- You need a function that can be called as part of a SQL statement and that must return a value that will be used in the statement.
- Your output needs to include a value for every input row or every group. For example:
SELECT MyFunction(col1) FROM table1;
- You need to perform simple queries with SQL, such as SELECT statements.

Usage and Behavior Differences

- UDFs return a value, stored procedures need not
- UDFs return values are directly usable in SQL; stored procedure return values may not be
- UDFs can be called in context of another statement; stored procedures are called independently
- Multiple UDFs may be called with one statement; a single stored procedure is called with one statement
- UDFs may access the database with simple queries only; stored procedures can execute DDL and DML statements

POP QUIZ:

Snowflake UDFs



What is the primary purpose of a UDF in Snowflake?

- A: To backup data
- B: To automate administrative tasks
- C: To perform custom calculations
- D: To manage user roles



POP QUIZ:

Snowflake UDFs



What is the primary purpose of a UDF in Snowflake?

- A: To backup data
- B: To automate administrative tasks
- C: To perform custom calculations**
- D: To manage user roles

Explanation: UDFs in Snowflake are designed to allow users to define custom functions for specific calculations or transformations that aren't provided by Snowflake's built-in functions.



POP QUIZ:

Snowflake UDFs



Which types of UDFs does Snowflake support? (select all that apply)

- A: Scalar UDFs
- B: Tabular UDFs
- C: Aggregate UDFs
- D: Windowed UDFs



POP QUIZ:

Snowflake UDFs



Which types of UDFs does Snowflake support? (select all that apply)

A: Scalar UDFs

B: Tabular UDFs

C: Aggregate UDFs

D: Windowed UDFs

Explanation: Snowflake supports Scalar UDFs, which return a single value, and Tabular UDFs, which return a table. Aggregate and Windowed UDFs are not explicitly categorized as such in Snowflake's UDF Support.



POP QUIZ:

Snowflake UDFs



Which languages can you use to write a UDF in Snowflake?

- A: Python
- B: SQL
- C: Java
- D: Go



POP QUIZ:

Snowflake UDFs



Which languages can you use to write a UDF in Snowflake?

A: Python

B: SQL

C: Java

D: Go



POP QUIZ:

Snowflake UDFs



What should you consider when deciding to use a UDF in Snowflake?
(Select all that apply)

- A: Reusability of the function across multiple queries
- B: Complexity of the logic not met by built-in functions
- C: User role administration
- D: Performance implications



POP QUIZ:

Snowflake UDFs



What should you consider when deciding to use a UDF in Snowflake? (Select all that apply)

A: Reusability of the function across multiple queries

B: Complexity of the logic not met by built-in functions

C: User role administration

D: Performance implications

Explanation: UDFs are great for reusability and for performing custom calculations not covered by built-in functions. However, it's essential to understand potential performance implications when using UDFs, especially in large datasets. User role administration is not directly related to the decision of using a UDF.



POP QUIZ:

Snowflake UDFs



If a UDF in Snowflake is defined using JavaScript, what type of UDF is it?

- A: SQL UDF
- B: Scalar UDF
- C: Tabular UDF
- D: JavaScript UDF



POP QUIZ:

Snowflake UDFs



If a UDF in Snowflake is defined using JavaScript, what type of UDF is it?

A: SQL UDF

B: Scalar UDF

C: Tabular UDF

D: JavaScript UDF

Explanation: In Snowflake, when a UDF is defined using JavaScript, it's referred to as a JavaScript UDF. The function can be either scalar or tabular, but the defining language make it a JavaScript UDF.



External Functions in Snowflake

- External functions call executable code that is developed, maintained, stored and executed outside Snowflake
- External functions extend Snowflake's capabilities
- Allow for Snowflake to integrate and interact with external services
- External functions bridge cloud platforms and Snowflake

Key Components: External Function

- Does not contain its own code, simply calls code that is stored and executed outside of Snowflake
- Database object - contains information that Snowflake uses to call the remote service

Key Components: Remote Service

- Remotely executed code known as remote service
- Remote service must be a function - it must return a value
- Snowflake supports *scalar* external functions; the remote service must return exactly one row for each row received.
- Remote service must accept JSON inputs and return JSON outputs
- Examples of remote service: AWS Lambda Function, Azure Function

Key Components: Proxy Service

- Snowflake does not call a remote service directly. Instead, Snowflake calls a proxy service, which relays the data to the remote service.
- The proxy service can increase security by authenticating requests to the remote service.
- The proxy service can support subscription-based billing for a remote service. For example, the proxy service can verify that a caller to the remote service is a paid subscriber.
- The proxy service also relays the response from the remote service back to Snowflake.
- Examples of proxy services include:
 - Amazon API Gateway.
 - Microsoft Azure API Management service.

Key Components: API Integration

- An *integration* is a Snowflake object that provides an interface between Snowflake and third-party services. An API integration stores information, such as security information, that is needed to work with a proxy service or remote service.
- An API integration is created with the CREATE API INTEGRATION command.

Understanding External Functions

- Functions that invoke an endpoint (API, AWS Lambda, Azure Functions, etc.)
- Allow Snowflake to integrate with external services for complex computations
- Return results from the external service back to Snowflake

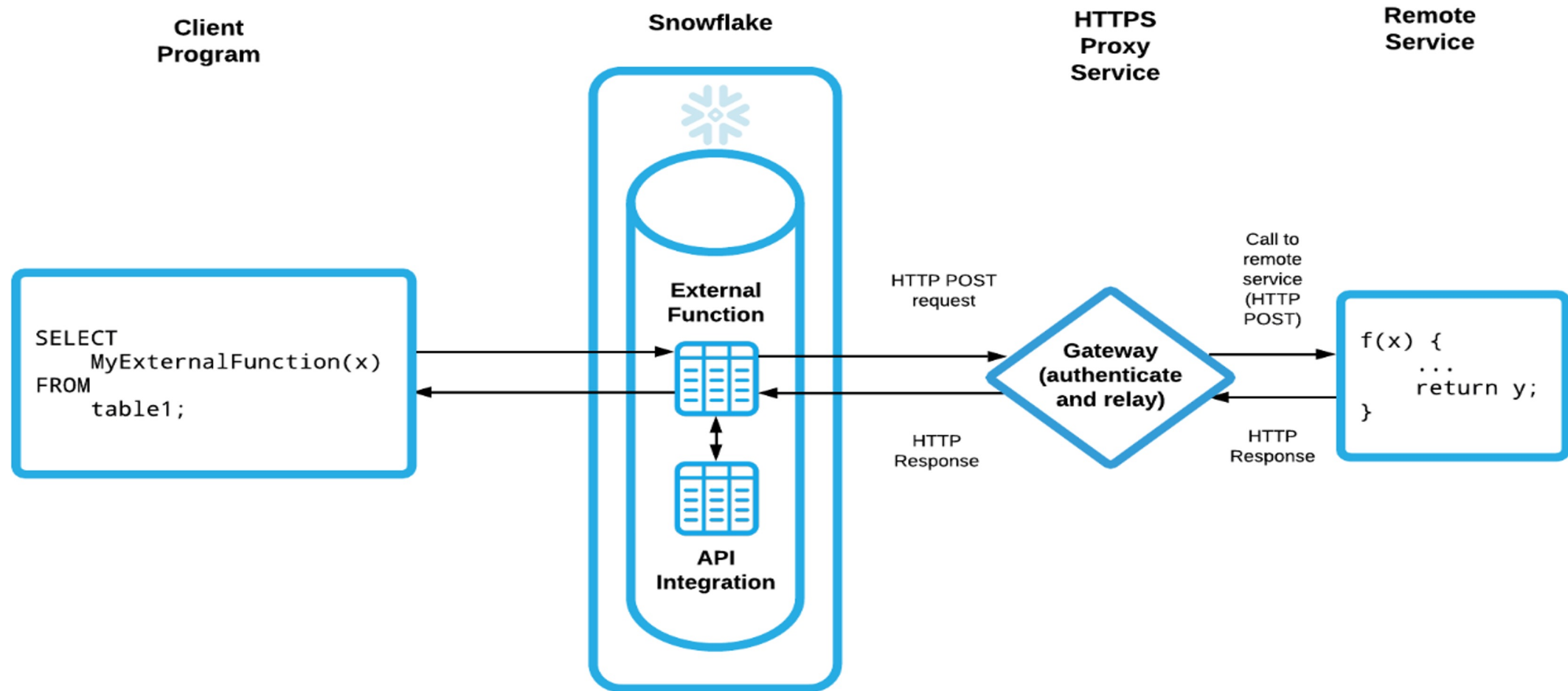
Key Components & Architecture

- API Integration: connects Snowflake with external services
- External Function Definition: Specifies the API Integration & other attributes
- Proxy Service: Acts as an intermediary between Snowflake and the external service

Benefits of Using External Functions

- Leverage specialized computation from external services
- Integrate with machine learning models, custom libraries, or third party tools
- Enhance data processing without moving data out of Snowflake

External Functions in Snowflake



Setting up External Functions

- Define the API integration using 'CREATE API INTEGRATION'
- Real-time data validation with third-party tools
- Custom transformations leveraging external libraries

Use Cases for External Functions

- Advanced analytics and machine learning scoring
- Specify the external service endpoint and desired API
- Create the external function using 'CREATE EXTERNAL FUNCTION'
- Code for remote service can be written in languages that other UDFs cannot be written in, including:
 - Go
 - C#
- Developers can write remote services that can be called both from Snowflake and from other software written to use the same interface.

Making the Most of External Functions

- Ensure robust error handling in external services
- Be cautious of latency; keep the external computation time-efficient
- Securely manage credentials and access permissions

POP QUIZ:

Snowflake External Functions



What is the main purpose of using external functions in Snowflake?

- A: To query Snowflake from an external service
- B: To process data using Snowflake's built-in functions only
- C: To offload compute-intensive tasks to external functions
- D: To handle Snowflake's user authentication



POP QUIZ:

Snowflake External Functions



What is the main purpose of using external functions in Snowflake?

A: To query Snowflake from an external service

B: To process data using Snowflake's built-in functions only

C: To offload compute-intensive tasks to external functions

D: To handle Snowflake's user authentication

Explanation: External functions in Snowflake allow computations to be offloaded to external (Like AWS or Azure Functions), providing a way to integrate Snowflake with other systems for specialized processing.



POP QUIZ:

Snowflake External Functions



Which of the following is a necessary component when setting up an external function in Snowflake?

- A: Snowflake warehouse
- B: Remote service
- C: Role hierarchy
- D: Backup function



POP QUIZ:

Snowflake External Functions



Which of the following is a necessary component when setting up an external function in Snowflake?

A: Snowflake warehouse

B: Remote service

C: Role hierarchy

D: Backup function

Explanation: An external function in Snowflake relies on a remote service (like AWS Lambda) to execute the logic of the function. The other options are not directly related to the setup of an external function.



POP QUIZ:

Snowflake External Functions



When setting up an external function in Snowflake, why might you define an API integration? (Select all that apply)

- A) To authenticate Snowflake to an external service.
- B) To increase the storage capacity of Snowflake.
- C) To specify allowed request and response formats.
- D) To securely transmit data between Snowflake and the external service.



POP QUIZ:

Snowflake External Functions



When setting up an external function in Snowflake, why might you define an API integration? (Select all that apply)

- A) To authenticate Snowflake to an external service.
- B) To increase the storage capacity of Snowflake.
- C) To specify allowed request and response formats.
- D) To securely transmit data between Snowflake and the external service.

Explanation: API integrations in Snowflake are used to securely authenticate and communicate with external services, ensuring that data is transmitted safely and in an authorized manner.



POP QUIZ:

Snowflake External Functions



Which cloud platforms' serverless functions can be utilized as remote services for Snowflake's external functions? (Select all that apply)

- A) AWS Lambda
- B) Google Cloud Functions
- C) Azure Functions
- D) IBM Cloud Functions



POP QUIZ:

Snowflake External Functions



Which cloud platforms' serverless functions can be utilized as remote services for Snowflake's external functions? (Select all that apply)

A) AWS Lambda

B) Google Cloud Functions

C) Azure Functions

D) IBM Cloud Functions



Transaction Management in Snowflake

Definition:

- Transaction management ensure the consistency, reliability, and correct sequencing of database operations

Importance in Snowflake:

- Enables ACID-compliant operations
- Ensures data integrity across complex operations
- Allows for the flexibility of manual or automatic control

Transactions in Snowflake

Auto Commit Mode:

- Each SQL statement is treated as a transaction
- Commits automatically on successful execution

Multi-statement Mode:

- Allows for bundling of several SQL operations into one transaction
- Requires manual COMMIT to save or ROLLBACK to undo

Starting a Transaction:

- Use 'START TRANSACTION' to begin
- Blocks other transactions from making conflicting modifications

Ending a Transaction:

- 'COMMIT': save all the changes
- 'ROLLBACK': Discard all the changes made within the transaction

Transaction Isolation Levels In Snowflake

READ COMMITTED (Default):

- Only sees data from other transactions that have been committed
- Avoid dirty reads

SERIALIZABLE:

- Full isolation. Operations in the transaction are invisible to other transactions until committed
- Avoids dirty reads, non-repeatable reads, and phantom reads.

Note: Snowflake does not use READ UNCOMMITTED or REPEATABLE READ levels, common in some other databases.

Transaction Conflicts & Resolutions

- Conflicts Occur When:
 - Two transactions try to modify the same object simultaneously
 - One transaction reads data another transaction is modifying
- Snowflake's Approach:
 - Uses optimistic concurrency control
 - If a conflict is detected, the later transaction is aborted

Monitoring Transactions

- System Views & Functions:
 - 'SHOW TRANSACTIONS': Display active transactions
 - 'SHOW LOCKS': View all current locks in the system
- Information Schema:
 - 'TRANSACTIONS' view: Detailed info on all past and current transactions
 - 'STATEMENTS' view: History and details of SQL statements

Best Practices in Snowflake Transaction Management

- Limit Transaction Duration: Reduces potential conflicts and resource locking
- Be Explicit: Always specify 'START TRANSACTION' and use 'COMMIT' or 'ROLLBACK' as appropriate
- Monitor Actively: Keep an eye on long-running or blocked transactions using Snowflake's monitoring tools

Experiment



Content
LAB 6 -
Transactions

Semi-structured Data in Snowflake

Definition:

- Data that doesn't conform to the traditional table-row-column structure
- Examples: JSON, Avro, Parquet, XML, ORC

Snowflake's Approach:

- Native support to store, retrieve, and process semi-structured data
- Enables seamless integration with structured data

Storage of Semi-structured Data

VARIANT Data Type:

- Snowflake's primary type for semi-structured data
- Can store up to 16 MB of data

Additional Data Types:

- 'OBJECT': For JSON objects
- 'ARRAY': For JSON arrays

Loading Semi-structured Data into Snowflake

Bulk Load:

- Use the 'COPY INTO' command
- Automatically recognizes and processes JSON, AVro, and Parquet, and ORC

File Format Options:

- Set specific properties like 'TYPE', 'STRIP_OUTTER_ARRAY', etc., to customize data loading

Querying Semi-structured Data

Dot Notation:

- Access data elements directly. E.g., 'SELECT myVraiant:propertyName'.

Flatten Function:

- Convert nested semi-structured data into individual rows
- Useful for data with varied and nested structures

Using Semi-structured Data with SQL Functions

CAST:

- Convert semi-structured data types to conventional SQL types

JSON Functions:

- 'TO_VARIANT', 'IS_OBJECT', 'IS_ARRAY', etc.
- These assist in navigating and extracting specific data from the semi-structured content

Performance Optimization for Semi-structured Data

Automatic Clustering:

- Snowflake automatically clusters semi-structured data for optimal query performance

Materialized Views:

- Create views on frequently queried portions of the semi-structured data
- Boosts performance for repeated querying

Integration with Structured Data

JOIN Operations:

- Semi-structured data can be joined with structured tables
- Offers a holistic approach to data analytics

Views and UDFs:

- Use views and user-defined functions to transform or represent semi-structured data in ways that can be integrated with structured queries

Best Practices for Semi-structured Data in Snowflake

Data Storage:

- Store in native format for flexibility

Data Retrieval:

- Use Snowflake's native functions for extraction rather than transforming data at the source

Schema Evolution:

- Use VARIANT types to handle change in data schema gracefully

Experiment



Content
LAB 7 -
Semistructured
Data

Snowpark Overview

Purpose of Snowpark API: An intuitive library to process data at scale directly in Snowflake.

Unique Feature: Develop and process data without moving it outside of Snowflake

Supported Languages: Java, Python, and Scala

Snowpark's Key Features

Benefits Over Spark Connector:

- Libraries and patterns purpose-built for different languages
- Write Snowpark code using local tools (e.g., Jupyter, VS Code)
- All operations, including UDFs, are pushed down to Snowflake's data cloud
- No need for an external cluster; all computation done in Snowflake

SQL Statement Construction with Snowpark:

- Snowpark offers programming language constructs for SQL statements
- Use native constructs (e.g., select method) for advantages like code completion and type checking

Advanced Snowpark Capabilities

- **Reduced Data Transfer:**

- Snowpark batches operations to reduce data transfer and improve performance.
- Core Abstraction: DataFrame, representing data and providing methods for operations.
- Lazily evaluate DataFrame objects, sending SQL statements to Snowflake for execution.

- **Inline UDF Creation:**

- UDFs can be created directly within a Snowpark app.
- Functions written in the client's language can be pushed to Snowflake to operate at scale.
- UDFs enable Snowflake to parallelize and apply logic at scale.

Snowpark ML

- **Preview Feature:** Currently open and available to all accounts.
- **Purpose:** Provides tools, SDKs, and infrastructure for building and deploying machine learning models within Snowflake.
- **Integration:** Works alongside Snowpark Python, enabling users to manage training/test data and prediction results through Snowpark DataFrames.
- **Platform Support:** Compatible with any IDE supporting Snowpark Python client applications.

Key Components of Snowpark ML

- **Snowpark ML Development:**

- Features Python APIs for efficient in-Snowflake model development.
- Offers the modeling package (`snowflake.ml.modeling`) for data preprocessing, feature engineering, and model training.
- Integrates familiar ML libraries: `scikit-learn`, `xgboost`, `lightgbm`.
- Upcoming framework connectors for Pytorch and Tensorflow in native loader formats.

- **Snowpark ML Ops:**

- Complements Development API by adding model management and Snowflake integration.
- Introduces the FileSet API for data materialization from Snowpark DataFrame to Snowflake internal stage.
- Features a model registry as a Python API for model management and deployment as UDFs in Snowflake warehouses.

Key Components of Snowpark ML

- **Snowpark ML Development:**

- Features Python APIs for efficient in-Snowflake model development.
- Offers the modeling package (snowflake.ml.modeling) for data preprocessing, feature engineering, and model training.
- Integrates familiar ML libraries: scikit-learn, xgboost, lightgbm.
- Upcoming framework connectors for Pytorch and Tensorflow in native loader formats.

- **Snowpark ML Ops:**

- Complements Development API by adding model management and Snowflake integration.
- Introduces the FileSet API for data materialization from Snowpark DataFrame to Snowflake internal stage.
- Features a model registry as a Python API for model management and deployment as UDFs in Snowflake warehouses.

Experiment



Content

Lab 8 - Machine
Learning with
Snowpark

POP QUIZ:

Snowpark



What is Snowpark primarily used for in Snowflake?

- a) Data visualization
- b) Web development
- c) Building data pipelines and UDFs
- d) Database administration



POP QUIZ:

Snowpark



What is Snowpark primarily used for in Snowflake?

- a) Data visualization
- b) Web development
- c) Building data pipelines and UDFs
- d) Database administration

Explanation: Snowpark is used to build data pipelines and user-defined functions (UDFs) within Snowflake.



POP QUIZ:

Semi-Structured Data



In Snowflake, how is semi-structured data typically stored?

- a) Rows and Columns
- b) VARIANT, OBJECT, and ARRAY columns
- c) Bitmaps
- d) Linked Lists



POP QUIZ:

Semi-Structured Data



In Snowflake, how is semi-structured data typically stored?

- a) Rows and Columns
- b) VARIANT, OBJECT, and ARRAY columns
- c) Bitmaps
- d) Linked Lists

Explanation: Snowflake supports the storage of semi-structured data in VARIANT, OBJECT, and ARRAY data types.



POP QUIZ:

Transaction Management



When performing a transaction in Snowflake, what command should be used to save the changes?

- a) COMMIT
- b) SAVE
- c) APPLY
- d) ACTIVATE



POP QUIZ:

Transaction Management



When performing a transaction in Snowflake, what command should be used to save the changes?

a) COMMIT

b) SAVE

c) APPLY

d) ACTIVATE

Explanation: The COMMIT command is used to save the changes made in a transaction in Snowflake



POP QUIZ:

Semi-Structured Data



Which of the following commands can be used to undo operations in a transaction in Snowflake?

- a) UNDO
- b) REJECT
- c) CANCEL
- d) ROLLBACK



POP QUIZ:

Semi-Structured Data



Which of the following commands can be used to undo operations in a transaction in Snowflake?

- a) UNDO
- b) REJECT
- c) CANCEL
- d) ROLLBACK

Explanation: The ROLLBACK command undoes operations within a transaction in Snowflake.



POP QUIZ:

Semi-Structured Data



In Snowflake, which datatype is best for storing JSON data?

- a) TEXT
- b) BLOB
- c) VARIANT
- d) INT



POP QUIZ:

Semi-Structured Data



In Snowflake, which datatype is best for storing JSON data?

- a) TEXT
- b) BLOB
- c) VARIANT**
- d) INT

Explanation: The VARIANT datatype is used for storing semi-structured data formats such as JSON in Snowflake.



POP QUIZ:

Semi-Structured Data



9. Which of the following is NOT a semi-structured data format supported by Snowflake?

- a) XML
- b) Parquet
- c) CSV
- d) Avro



POP QUIZ:

Semi-Structured Data



9. Which of the following is NOT a semi-structured data format supported by Snowflake?

- a) XML
- b) Parquet
- c) CSV
- d) Avro

Explanation: While Snowflake supports various semi-structured data formats, CSV is typically considered structured.



POP QUIZ:

Mock Exam Questions



A Data Engineer is preparing to load staged data from an external stage using a task object

Which of the following practices will provide the MOST efficient load performance?

- a) Store the files on the external stage to ensure caching is maintained
- b) PUT all files in a single directory
- c) Limit file names to under 30 characters
- d) Organize files into logical paths that reflect a scheduling pattern



POP QUIZ:

Mock Exam Questions



A Data Engineer is preparing to load staged data from an external stage using a task object

Which of the following practices will provide the MOST efficient load performance?

- a) Store the files on the external stage to ensure caching is maintained
- b) PUT all files in a single directory
- c) Limit file names to under 30 characters
- d) Organize files into logical paths that reflect a scheduling pattern



POP QUIZ:

Mock Exam Questions



A Data Engineer is working on a project that requires data to be moved directly from an internal stage to an external stage

Which of the following is the QUICKEST way to accomplish this

- a) `COPY INTO @myExtStage from (Select $1, $2, ... @myInternalStage);`
- b) Copy the data from the internal stage to a table and then unload the data to an external stage
- c) `COPY INTO @myExtStage from @myInternalStage;`
- d) Write a custom script to move the data



POP QUIZ:

Mock Exam Questions



A Data Engineer is working on a project that requires data to be moved directly from an internal stage to an external stage

Which of the following is the QUICKEST way to accomplish this

- a) `COPY INTO @myExtStage from (Select $1, $2, ... @myInternalStage);`
- b) Copy the data from the internal stage to a table and then unload the data to an external stage
- c) `COPY INTO @myExtStage from @myInternalStage;`
- d) Write a custom script to move the data



POP QUIZ:

Mock Exam Questions



The s1 schema contains two permanent tables that were created as shown below:

```
CREATE TABLE table_a (c1 INT)  
DATA_RETENTION_TIME_IN_DAYS = 10;  
CREATE TABLE table_b (c1 INT);
```

What will be the impact of running the following command?

```
ALTER SCHEMA S1 SET  
DATA_RETENTION_TIME_IN_DAYS = 20;
```

- a) The retention time on table_a does not change; table_b is set to 20 days.
- b) An error will be generated, a data retention time on a schema cannot be set
- c) The retention time on both tables will be set to 20 days
- d) The retention time will not change on either table

POP QUIZ:

Mock Exam Questions



The s1 schema contains two permanent tables that were created as shown below:

```
CREATE TABLE table_a (c1 INT)  
DATA_RETENTION_TIME_IN_DAYS = 10;  
CREATE TABLE table_b (c1 INT);
```

What will be the impact of running the following command?

```
ALTER SCHEMA S1 SET  
DATA_RETENTION_TIME_IN_DAYS = 20;
```

- a) The retention time on table_a does not change; table_b is set to 20 days.
- b) An error will be generated, a data retention time on a schema cannot be set
- c) The retention time on both tables will be set to 20 days
- d) The retention time will not change on either table

POP QUIZ:

Mock Exam Questions



What are the steps to load data from an archival cloud storage class (for example, Amazon S3 Glacier or Microsoft Azure Archive Storage) into Snowflake?

- A. Same steps as loading data from a non-archival cloud storage class.
- B. You should create an ARCHIVAL stage; then, you can perform the same steps as loading data from a non-archival cloud storage class.
- C. It's not possible to do it directly on Snowflake



POP QUIZ:

Mock Exam Questions



What are the steps to load data from an archival cloud storage class (for example, Amazon S3 Glacier or Microsoft Azure Archive Storage) into Snowflake?

- A. Same steps as loading data from a non-archival cloud storage class.
- B. You should create an ARCHIVAL stage; then, you can perform the same steps as loading data from a non-archival cloud storage class.
- C. It's not possible to do it directly on Snowflake

POP QUIZ:

Mock Exam Questions



After creating the table mytable, we execute the following commands: `CREATE STREAM MYSTREAM ON TABLE MYTABLE;`

`INSERT INTO MYTABLE VALUES (15);`

What will be the output after executing the following command?

`SELECT SYSTEM$STREAM_HAS_DATA('MYSTREAM');`

- A. It will return "True".
- B. It will return "False".
- C. It will return "15".
- D. It will return "Null".

POP QUIZ:

Mock Exam Questions



After creating the table mytable, we execute the following commands: `CREATE STREAM MYSTREAM ON TABLE MYTABLE;`

`INSERT INTO MYTABLE VALUES (15);`

What will be the output after executing the following command?

`SELECT SYSTEM$STREAM_HAS_DATA('MYSTREAM');`

- A. It will return "True".
- B. It will return "False".
- C. It will return "15".
- D. It will return "Null".

POP QUIZ:

Mock Exam Questions



Does an external function's identifier need to be unique for the schema in which the function is created?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



Does an external function's identifier need to be unique for the schema in which the function is created?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



Does an external function's identifier need to be unique for the schema in which the function is created?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



Which of the following JOIN statements are supported by Snowflake?

- A. Inner join
- B. Outer join
- C. Cross join
- D. Natural join
- E. All of the above



POP QUIZ:

Mock Exam Questions



Which of the following JOIN statements are supported by Snowflake?

- A. Inner join
- B. Outer join
- C. Cross join
- D. Natural join
- E. All of the above



POP QUIZ:

Mock Exam Questions



After creating a database and a schema using the following commands:

```
CREATE OR REPLACE DATABASE MY_DB  
DATA_RETENTION_TIME_IN_DAYS=30;
```

```
CREATE OR REPLACE SCHEMA S1  
DATA_RETENTION_TIME_IN_DAYS=50;
```

How Long will we be able to access the data from the schema using the Time Travel functionality if we drop the database?

- A. The default time on the account
- B. 30 days
- C. 50 days
- D. 90 days



POP QUIZ:

Mock Exam Questions



After creating a database and a schema using the following commands:

```
CREATE OR REPLACE DATABASE MY_DB  
DATA_RETENTION_TIME_IN_DAYS=30;
```

```
CREATE OR REPLACE SCHEMA S1  
DATA_RETENTION_TIME_IN_DAYS=50;
```

How Long will we be able to access the data from the schema using the Time Travel functionality if we drop the database?

- A. The default time on the account
- B. 30 days
- C. 50 days
- D. 90 days



POP QUIZ:

Mock Exam Questions



If you want your Data warehouse available with no lag time, is it a good idea to consider disabling the auto-suspend functionality?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



If you want your Data warehouse available with no lag time, is it a good idea to consider disabling the auto-suspend functionality?

- A. True
- B. False

Explanation: You might consider disabling this functionality in two cases; if you have a heavy, steady workload; and if you require the warehouse always to be available with no delay time. However, disabling this might incur more compute costs, so you should consider the pros and cons.



POP QUIZ:

Mock Exam Questions



Our company wants to execute a complicated ETL process inside Snowflake every day on the new data that is coming. What is the best way to automate it?

- A. Create an external script that runs every day at the same hour using AWS Lambda.
- B. Create a Snowflake task.
- C. Create a docker image and use it as an external function



POP QUIZ:

Mock Exam Questions



Our company wants to execute a complicated ETL process inside Snowflake every day on the new data that is coming. What is the best way to automate it?

- A. Create an external script that runs every day at the same hour using AWS Lambda.
- B. **Create a Snowflake task.**
- C. Create a docker image and use it as an external function

Explanation: The code runs inside Snowflake; it's as easy as creating a task to run it daily at a specific hour.

POP QUIZ:

Mock Exam Questions



Which is generally the slowest option when using the COPY INTO command?

- A. By path for internal stages and prefix for external
- B. Specifying a list of files to load
- C. Using pattern matching



POP QUIZ:

Mock Exam Questions



Which is generally the slowest option when using the COPY INTO command?

- A. By path for internal stages and prefix for external
- B. Specifying a list of files to load
- C. Using pattern matching

Explanation: These are the three options the COPY INTO command supports for loading data. Specifying a list of files to load is the fastest one, and using pattern matching is usually the slowest.



Performance Optimization

- **Exploring Execution Times**

- Gain historical insights into query performance.
- Use Snowflake's web interface or query ACCOUNT_USAGE schema

.

- **Optimizing Warehouses for Performance**

- Fine-tune computing power for better query performance.
- Strategies include enabling Query Acceleration Service

- **Optimizing Storage for Performance**

- Store similar data together for efficiency.
- Create optimized data structures.
- Define specialized data sets for improved query performance.
- Helps in choosing between Automatic Clustering, Search Optimization Service, and materialized views.

Viewing Execution Times and Load in Snowflake with Snowsight

1. Viewing Queries:

- Access: Sign into Snowsight → Activity → Query History.
- Insights:
 - Use the Duration column to gauge execution time.
 - Sort by duration to find longest-running queries.
 - Filter by user or warehouse for focused analysis.

2. Analyzing Warehouses:

- Access: Sign into Snowsight → Admin → Warehouses.
- Insights:
 - Select a warehouse to analyze.
 - Utilize the Warehouse Activity chart to view load and query queues.

3. Assessing Tasks:

- Access: Sign into Snowsight → Activity → Task History.
- Insights:
 - Review the execution duration of tasks' SQL code.

Key Takeaway:

Leverage Snowsight for visual and detailed insights into the performance of queries, warehouse load, and task execution times in Snowflake.

Drilling Down into Execution Time with Query Profile

Understanding Query Profile:

- Purpose: Allows detailed analysis of query execution.
- Key Feature: "Most Expensive Nodes" pane identifies longest-executing operator nodes.
- Depth: Drill down into execution time spent in specific categories of query processing.

Accessing Query Profile:

Sign In: Access Snowsight.

Navigate: Select Activity → Query History.

Select Query: Choose the query ID you want to analyze.

Analyze: Click the Query Profile tab for detailed insights.

Tip:

- Programmatic Access: Use `GET_QUERY_OPERATOR_STATS` function to fetch performance statistics of the Query Profile programmatically.

Key Takeaway:

Utilize the Query Profile in Snowsight for a granular breakdown of query execution times, pinpointing the most time-consuming parts of a query.

Writing Queries to Explore Execution Times in Snowflake

Scope: Analyze query, task execution times, and warehouse load

Data Source: ACCOUNT_USAGE schema (default access by ACCOUNTADMIN role).

Alternative: Information Schema QUERY_HISTORY for users without ACCOUNT_USAGE access.

Latency: ACCOUNT_USAGE views have a delay in updates.

Query Performance

Query: Top n Longest-Running Queries

Purpose: Identify the top 50 longest-running queries in the last day.

Adjustment: Modify DATEADD function for desired time period.

Sample Query:

```
SELECT query_id,  
       ROW_NUMBER() OVER(ORDER BY partitions_scanned DESC) AS query_id_int,  
       query_text,  
       total_elapsed_time/1000 AS query_execution_time_seconds,  
       partitions_scanned,  
       partitions_total,  
FROM snowflake.account_usage.query_history Q  
WHERE warehouse_name = 'my_warehouse' AND TO_DATE(Q.start_time) > DATEADD(day,-1,TO_DATE(CUR  
      AND total_elapsed_time > 0 --only get queries that actually used compute  
      AND error_code IS NULL  
      AND partitions_scanned IS NOT NULL  
ORDER BY total_elapsed_time desc  
LIMIT 50;
```

 Copy

Query Performance

Query: Queries Organized by Execution Time Over Past Month

Purpose: Group queries by execution time buckets for warehouse performance assessment.

Sample Query:

```
SELECT
  CASE
    WHEN Q.total_elapsed_time <= 60000 THEN 'Less than 60 seconds'
    WHEN Q.total_elapsed_time <= 300000 THEN '60 seconds to 5 minutes'
    WHEN Q.total_elapsed_time <= 1800000 THEN '5 minutes to 30 minutes'
    ELSE 'more than 30 minutes'
  END AS BUCKETS,
  COUNT(query_id) AS number_of_queries
FROM snowflake.account_usage.query_history Q
WHERE TO_DATE(Q.START_TIME) > DATEADD(month,-1,TO_DATE(CURRENT_TIMESTAMP()))
  AND total_elapsed_time > 0
  AND warehouse_name = 'my_warehouse'
GROUP BY 1;
```

 Copy

Query Performance

Warehouse Load:

Query: Total Warehouse Load

Purpose: Evaluate the total load of a warehouse for executed and queued queries.

Sample Query:

```
SELECT TO_DATE(start_time) AS date,  
       warehouse_name,  
       SUM(avg_running) AS sum_running,  
       SUM(avg_queued_load) AS sum_queued  
FROM snowflake.account_usage.warehouse_load_history  
WHERE TO_DATE(start_time) >= DATEADD(month,-1,CURRENT_TIMESTAMP())  
GROUP BY 1,2  
HAVING SUM(avg_queued_load) >0;
```

Query Performance

Task Performance:

Query: Longest Running Tasks

Purpose: Identify the longest-running tasks in the last day for optimization.

Sample Query:

```
SELECT DATEDIFF(seconds, query_start_time, completed_time) AS duration_seconds,*  
FROM snowflake.account_usage.task_history  
WHERE state = 'SUCCEEDED'  
      AND query_start_time >= DATEADD (day, -1, CURRENT_TIMESTAMP())  
ORDER BY duration_seconds DESC;
```

Query Performance - Key Points

1. **Custom Reports:** Tailor queries for custom performance reports and dashboards.
2. **Focus Areas:** Query performance, warehouse load, and task execution times.
3. **Data Freshness:** Consider latency in ACCOUNT_USAGE views for real-time analysis needs.

KEY POINTS



Optimizing Warehouse Performance in Snowflake

Overview

- Objective: Enhance query performance through warehouse tuning.
- Approach: Implement strategies focused on compute resource optimization.

Strategies:

1. Reduce queues
2. Resolve memory spillage
3. Increase warehouse size
4. Try query Acceleration
5. Optimize Warehouse Cache
6. Limit Concurrently Running Queries

Tips & Considerations

- Workload Uniformity: Optimization is more effective for similar workloads. Diverse queries might not benefit equally from certain optimizations.
- Test & Monitor: Test changes by re-running queries and monitoring execution times.
- Comprehensive Approach: Combine warehouse strategies with data storage optimizations for best results.

Reduce Queues

Diagnostic Tools:

- Snowsight: Check Warehouse Activity for queued load patterns
- SQL Query: Identify warehouses with queues in the last month

Key Strategies:

Distribute Queries

- Regular Warehouses: Create additional warehouses
- Address Usage Spikes: Relocate spike-causing queries

Adopt Multi-Cluster Warehouses

- Ideal for demand spikes (Requires Enterprise Edition)

Increase Maximum Clusters

- For existing multi-cluster setups

Cost Considerations:

- Be mindful of credit consumption for multi-cluster warehouses
- Economy scaling policy: Balances cost and elasticity

Configuration Steps:

- Regular Warehouses: Use Snowsight or CREATE WAREHOUSE command
- Multi-Cluster Warehouses: Edit warehouse settings in Snowsight

Resolve Memory Spillage

Objective: Enhance query performance by addressing memory spillage issues

Identify Problematic Queries:

- Diagnostic Query: Top 10 queries spilling to local and remote storage
- Database: snowflake.account_usage.query_history

Key Strategies:

Convert to Snowpark-Optimized Warehouse

- Offers 16x more memory per node, 10x local cache
- Suitable for queries needing more memory without added compute

Analyze Query Profile

- Identify operation nodes causing spillage

Cost Comparison:

- Standard vs. Snowpark-Optimized Warehouses
- X-Small to 6X-Large: 1 to 512 credits for Standard, 6 to 768 credits for Snowpark-Optimized (Medium to 6

Conversion Steps:

- Snowsight:
 - Sign in, navigate to Admin » Warehouses
 - Suspend warehouse
 - Edit and select Snowpark-Optimized
 - Save changes
- SQL:
 - ALTER WAREHOUSE SUSPEND;
 - ALTER WAREHOUSE my_analytics_wh SET WAREHOUSE_TYPE='snowpark-optimized';

Increase Warehouse Size

Objective: Improve query performance by adjusting warehouse size

Key Consideration: Larger warehouses offer more compute resources

When to Upsize:

- For larger, complex queries (small queries may not benefit as much)
- When current warehouse is not heavily loaded

Diagnostic Approach:

- Query: Warehouse Load
 - Measures load for executed and queued queries
 - Helps determine if upsize can boost performance

Cost Implications:

- Increased warehouse size = Higher credit consumption
- Credits range from 1/hour (X-Small) to 512/hour (6X-Large)

Cost-Benefit Analysis:

- Upsizing may offset costs if it significantly reduces query execution time

Best Practices:

Restrict permissions for adjusting warehouse size
Ensure warehouses are resized back to original after specific queries

Query Acceleration Service

Overview:

- Purpose: Enhance Snowflake's performance by efficiently processing demanding queries.
- How it Works: Offloads parts of query processing to shared compute resources.

Benefits of Query Acceleration

- Performance Boost: Reduces the impact of resource-intensive outlier queries.
- Optimized Workloads: Especially beneficial for:
 - Ad hoc analytics
 - Queries with unpredictable data volumes
 - Large data scans with selective filters

Identifying Queries for Acceleration

- Tools in Snowflake:
 - `SYSTEM$ESTIMATE_QUERY_ACCELERATION` function: Estimates performance gains for specific queries.
 - `QUERY_ACCELERATION_ELIGIBLE` View: Identifies eligible queries & warehouses.
- Some Queries Might Be Ineligible: For reasons such as non-selective filters, high cardinality, or insufficient partitions.



Understanding Scale Factor In Query Acceleration

Purpose of Scale Factor:

- The scale factor serves as a cost control mechanism, setting an upper limit on the amount of additional compute resources a warehouse can lease for query acceleration.

How It Works:

- It's a multiplier based on the warehouse size and cost, determining the extent of additional resources that can be leased.

Cost Implications:

- The cost of leasing these additional resources is calculated based on the scale factor and the base cost of the warehouse size.
- Importantly, the cost is consistent regardless of the number of queries using the service simultaneously.
- It's billed by the second, only when in use, and separately from standard warehouse usage.

Resource Utilization:

- Not every query will require the full extent of resources made available by the scale factor.
- The actual resource allocation depends on the query's eligibility for acceleration and the data volume involved.
- Even if a high scale factor is set, the service only uses as many resources as needed and available at the time.

Constraints:

- The availability of compute resources for query acceleration is subject to the availability of resources in the service and the number of other concurrent requests.

Default and Unlimited Settings:

- If not explicitly set, the scale factor defaults to 8.
- Setting the scale factor to 0 removes the upper limit, allowing queries to lease as many resources as necessary and available.

Query Acceleration - Continued

Cost Considerations

Separate Billing: Serverless compute resources are billed independently from the warehouse.

Cost Control: Scale factor sets a limit on serverless compute usage.

- Example: Scale factor of 5 means the serverless compute cost can't exceed 5 times the warehouse's rate.

Cost-Effective: Only queries that need extra compute power incur additional costs.

Enabling Query Acceleration

Requirement: Access to the shared SNOWFLAKE database.

Command:

- For maximized performance (no cost consideration):

```
ALTER WAREHOUSE my_wh SET  
ENABLE_QUERY_ACCELERATION = true  
QUERY_ACCELERATION_MAX_SCALE_FACTOR = 0;
```

Summary

- Query Acceleration is a powerful feature in Snowflake's Enterprise Edition to enhance query performance.
- Identifying the right candidates for acceleration is key.
- Cost considerations are essential; the scale factor plays a crucial role in controlling expenses.
- Enabling the service requires specific commands and settings.

Optimizing Warehouse Cache

Objective: Improve query performance by optimizing a warehouse's cache.

Benefit: A running warehouse maintains a cache of table data, speeding up subsequent queries.

Identifying Data Scanned from Cache

Determine the percentage of data scanned from cache to assess potential performance gains.

```
SELECT warehouse_name
, COUNT(*) AS query_count
, SUM(bytes_scanned) AS bytes_scanned
, SUM(bytes_scanned*percentage_scanned_from_cache) AS bytes_scanned_from_cache
, SUM(bytes_scanned*percentage_scanned_from_cache) / SUM(bytes_scanned) AS percent_scanned_
FROM snowflake.account_usage.query_history
WHERE start_time >= dateadd(month,-1,current_timestamp())
AND bytes_scanned > 0
GROUP BY 1
ORDER BY 5;
```

About the Cache and Auto-suspension

Auto-suspension Impact: The cache is dropped when the warehouse is suspended, affecting performance.

Recommendations:

- Tasks: Immediate suspension
- DevOps, DataOps, Data Science: Approximately 5 minutes
- BI, SELECT Queries: At least 10 minutes to maintain the cache

Optimizing Warehouse Cache - Continued

Cost Considerations

Running Warehouse Costs: A running warehouse consumes credits, even when idle.

Match Workload: Align auto-suspension setting with query frequency to avoid unnecessary costs and cache drops.

Configuring Auto-Suspension

Via Snowsight:

- Navigate to Admin » Warehouses.
- Find the warehouse, select Edit.
- Turn on Auto Suspend.
- Set Suspend After (min).
- Save changes.

Via SQL:

- Use ALTER WAREHOUSE command.
- Example: ALTER WAREHOUSE my_wh SET AUTO_SUSPEND = 600; (in seconds).

Summary

- Warehouse cache optimization can significantly improve query performance.
- Analyzing data scanned from cache helps identify optimization opportunities.
- Auto-suspension settings are crucial in maintaining the cache and optimizing costs.
- Regularly review and adjust auto-suspension settings based on query patterns and workload.

Limiting Concurrently Running Queries in Snowflake

Overview:

Objective: Improve the performance of queries by reducing concurrency.

Challenge: Queries running concurrently in a warehouse share resources, which can impact performance.

Solution: Use `MAX_CONCURRENCY_LEVEL` to limit concurrent queries

Impact of Lowering Concurrency

Benefits:

- Potentially more resources per query.
- Enhanced performance for large, complex, or multi-statement queries.

Considerations:

- Lowering concurrency may increase queue times for other queries.
- Requires testing to ensure desired outcomes.
- Alternative strategies: Dedicated warehouses, Query Acceleration Service.

Queue Management

Queueing Effects: Lower concurrency can lead to increased queuing.

Timeout Strategy: Adjust `STATEMENT_QUEUED_TIMEOUT_IN_SECONDS` to prevent extended queuing.

Limiting Concurrently Running Queries in Snowflake- Continued

Setting MAX_CONCURRENCY_LEVEL

Default: The default maximum concurrency level is 8.

Lowering Concurrency:

- Use the ALTER WAREHOUSE command.
- Specify a lower number for MAX_CONCURRENCY_LEVEL.
- Example: ALTER WAREHOUSE my_wh SET MAX_CONCURRENCY_LEVEL = 4;

Best Practices

Testing: Thoroughly test adjustments to ensure they positively impact performance.

Balancing Act: Find the right balance between concurrency and performance.

Monitor Queue Times: Regularly monitor the effect on queue times and adjust as necessary.

Summary

- Limiting concurrently running queries can enhance individual query performance.
- Consider alternative strategies for handling large or complex queries.
- Be mindful of the trade-offs, including increased queuing.
- Regular adjustments and monitoring are key to maintaining optimal performance.

Experiment



Content
LAB 9 - Query
Acceleration

Micro Partitions and Data Clustering in Data Warehousing

Traditional Data Warehouses: Static Partitioning

- Approach: Large tables are divided into static partitions.
- Purpose: Achieve acceptable performance and better scaling.
- Management: Partitions manipulated independently using specialized DDL and syntax.
- Limitations:
 - Maintenance Overhead: Time and resource-intensive to manage.
 - Data Skew: Results in disproportionately-sized partitions, affecting performance.

Snowflake Data Platform: Micro-partitioning

- Approach: Implements a unique form of partitioning called micro-partitioning.
- Advantages over Static Partitioning:
 - Reduces Maintenance Overhead: Easier management compared to traditional static partitioning.
 - Addresses Data Skew: Avoids issues of disproportionately-sized partitions.
 - Additional Benefits: Provides significant improvements not available in traditional systems.

Key Takeaway:

- Innovation in Partitioning: Snowflake's micro-partitioning offers a powerful and efficient alternative to static partitioning, overcoming its limitations and enhancing data warehousing capabilities.

Understanding Micro Partitions in Snowflake

What are Micro-partitions?

- Definition: Contiguous units of storage in Snowflake tables.
- Size: Each contains 50 MB to 500 MB of uncompressed data (compressed when stored).
- Organization: Groups of rows in tables mapped into individual micro-partitions, organized in a columnar fashion.

Advantages of Micro-partitioning

- Granular Pruning: Allows for extremely granular pruning of large tables, which may consist of millions of micro-partitions.
- Efficient Data Handling: Enhances the handling and querying of vast amounts of data.

Metadata Stored in Micro-partitions

- Range of Values: For each column in the micro-partition.
- Distinct Values: Number of distinct values.
- Additional Properties: Used for optimization and efficient query processing.

Automatic and Transparent Process

- Automatic Partitioning: All Snowflake tables are automatically micro-partitioned.
- Data Ordering: Tables are partitioned using the ordering of data as it is inserted/loaded.

Key Takeaway:

- Innovative Data Storage: Snowflake's micro-partitioning offers a highly efficient, granular, and automatic way of data storage and retrieval, revolutionizing the way large datasets are managed.

Benefits of Micro-Partitions in Snowflake

Automated Partitioning

Key Point: Micro-partitions are derived automatically, requiring no explicit definition or maintenance by users.

Advantage: Reduces manual workload and complexity.

Size and Efficiency

Size: Micro-partitions are small (50 to 500 MB, before compression).

Impact: Enables extremely efficient Data Manipulation Language (DML) operations and fine-grained pruning for faster queries.

Uniformity and Skew Prevention

Overlap in Ranges: Micro-partitions can overlap in their value ranges.

Uniform Size: Small and uniform size helps prevent data skew.

Benefits of Micro-Partitions in Snowflake - Continued

Columnar Storage and Scanning

Storage: Columns are stored independently within micro-partitions (columnar storage).

Query Scanning: Only columns referenced by a query are scanned, enhancing efficiency.

Individual Column Compression

Compression Approach: Columns are compressed individually within micro-partitions.

Optimization: Snowflake automatically determines the most efficient compression algorithm for each micro-partition.

Key Takeaway:

Enhanced Data Management: Snowflake's micro-partitioning offers a suite of benefits, including automation, size efficiency, skew prevention, and optimized storage, fundamentally improving data management and query performance.

Impact of Micro-Partitions on DML and Query Pruning

DML Operations (e.g. DELETE, UPDATE, MERGE)

- Key Point: Leverage underlying micro-partition metadata for table maintenance.
- Example: Some operations like deleting all rows from a table are metadata-only operations, simplifying the process.

Query Pruning and Efficiency

- Role of Metadata: Enables precise pruning of columns in micro-partitions during query run-time, including semi-structured data.
- Example: For a query accessing 10% of the values in a range, ideally only 10% of the micro-partitions are scanned.

Columnar Scanning for Enhanced Performance

- Scenario: Consider a large table with one year of historical data and date/hour columns.
- Query Targeting: A query targeting a particular hour would ideally scan 1/8760th of the micro-partitions, and only scan the portion containing the data for that hour.
- Advantage: Snowflake's columnar scanning ensures only relevant micro-partitions are scanned.

Impact of Micro-Partitions on DML and Query Pruning - Continued

Pruning Efficiency Ratio

- Optimal Pruning: The closer the ratio of scanned micro-partitions and columnar data is to the actual data selected, the more efficient the pruning.

Time-Series Data and Sub-Second Response

- Potential: Enables sub-second response times for queries within fine-grained ranges, such as one hour or less.

Key Takeaway:

- Optimized Data Handling: Micro-partitions significantly optimize DML operations and query pruning, leading to enhanced performance, especially for time-series data and queries with precise filters.

Data Clustering

What is Data Clustering?

- Definition: Sorting/ordering of data stored in tables along natural dimensions (e.g., date, geographic regions).
- Importance: Clustering is crucial for query performance, especially in very large tables.

Clustering in Snowflake

- Process: As data is inserted/loaded, clustering metadata is collected for each micro-partition.
- Utilization: This metadata is used to avoid unnecessary scanning of micro-partitions during querying.

DATA CLUSTERING

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
	2	4	3																									
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
	A	C	C																									
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
	UK	SP	DE																									
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
	11/2	11/2	11/2																									
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										

Clustering Information for MicroPartitions in Snowflake

1. Clustering Metadata Overview

- i. Total Micro-partitions: Number comprising the table.
- ii. Overlapping Micro-partitions: Number with values overlapping in specified columns.
- iii. Overlapping Depth: Depth of these overlapping micro-partitions.

2. Clustering Depth

- i. Definition: Average depth (1 or greater) of overlapping micro-partitions for specified columns in a table.
- ii. Indicator: Smaller average depth signifies better clustering for the specified columns.

3. Uses of Clustering Depth

- i. Monitoring: Assess the clustering "health" of a large table, especially over time with DML operations.
- ii. Decision Making: Determine if a large table could benefit from an explicitly defined clustering key.

4. Clustering Depth in Unpopulated Tables

- i. Note: A table with no micro-partitions (unpopulated/empty) has a clustering depth of 0.

5. Understanding Clustering Depth

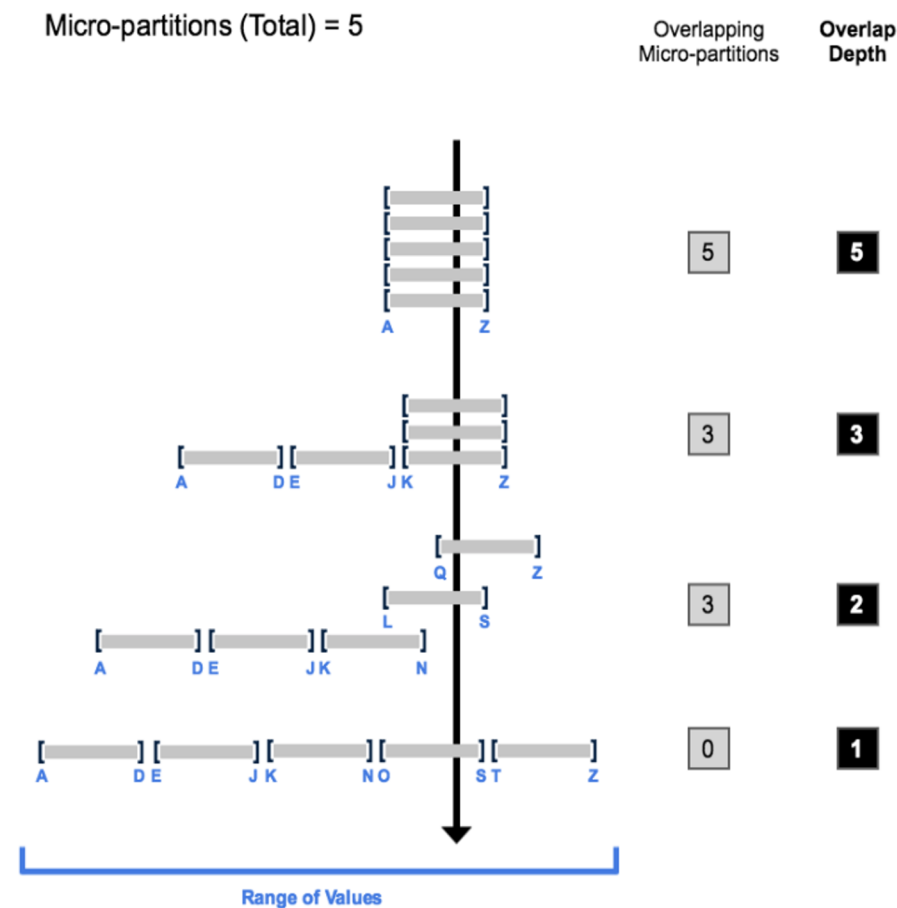
- i. Not Absolute: Clustering depth isn't a precise measure of a table's clustering effectiveness.
- ii. Best Indicator: Query performance remains the best indicator of how well-clustered a table is.
 - 1. Well-Clustered: If queries perform as needed or expected.
 - 2. Needs Clustering: If query performance degrades over time.

6. Key Takeaway:

- i. Practical Implications: Understanding and monitoring clustering depth helps in optimizing query performance and determining the need for clustering adjustments in Snowflake.

Clustering Depth Illustrated

The following diagram provides a conceptual example of a table consisting of five micro-partitions with values ranging from A to Z, and illustrates how overlap affects clustering depth:



As this diagram illustrates:

1. At the beginning, the range of values in all the micro-partitions overlap.
2. As the number of overlapping micro-partitions decreases, the overlap depth decreases.
3. When there is no overlap in the range of values across all micro-partitions, the micro-partitions are considered to be in a *constant state* (i.e. they cannot be improved by clustering).

The diagram is not intended to represent an actual table. In an actual table, with data contained in a large numbers of micro-partitions, reaching a constant state across all micro-partitions is neither likely nor required to improve query performance.

SYSTEM\$CLUSTERING_DEPTH

Computes the average depth of the table according to the specified columns (or the clustering key defined for the table). The average depth of a populated table (i.e. a table containing data) is always 1 or more. The smaller the average depth, the better clustered the table is with regards to the specified columns.

```
SYSTEM$CLUSTERING_DEPTH( '<table_name>' , '( <col1> [ , <col2> ... ] )' [ , '<predicate>' ]
```

Column(s) in the table used to calculate the clustering depth:

- For a table with no clustering key, this argument is required. If this argument is omitted, an error is returned.
- For a table with a clustering key, this argument is optional; if the argument is omitted, Snowflake uses the defined clustering key to calculate the depth.

Note

You can use this argument to calculate the depth for any columns in the table, regardless of the clustering key defined for the table.

SYSTEM\$CLUSTERING_DEPTH – Example

Calculate the clustering depth for a table using the clustering key defined for the table:

```
SELECT SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS');
```

```
+-----+  
| SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS') |  
+-----+  
| 2.4865                                |  
+-----+
```

SYSTEM\$CLUSTERING_DEPTH – Example

Calculate the clustering depth for a table using two columns in the table:

```
SELECT SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS', '(C2, C9)');
```

```
+-----+  
| SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS', '(C2, C9)') |  
+-----+  
| 23.1351 |  
+-----+
```

SYSTEM\$CLUSTERING_DEPTH – Example

Same as the previous example, but with a predicate on one of the columns:

```
SELECT SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS', '(C2, C9)', 'C2 = 25');
```

```
+-----+  
| SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS', '(C2, C9)') |  
+-----+  
| 11.2452 |  
+-----+
```

SYSTEM\$CLUSTERING_DEPTH – Example

Same as the previous example, but with a predicate on one of the columns:

```
SELECT SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS', '(C2, C9)', 'C2 = 25');
```

```
+-----+  
| SYSTEM$CLUSTERING_DEPTH('TPCH_ORDERS', '(C2, C9)') |  
+-----+  
| 11.2452 |  
+-----+
```


SYSTEM\$CLUSTERING_INFORMATION

Returns clustering information, including average clustering depth, for a table based on one or more columns in the table.

Syntax

```
SYSTEM$CLUSTERING_INFORMATION( '<table_name>'
                                [ , { '( <expr1> [ , <expr2> ... ] )' | <number_of_errors> } ] )
```

Column names or expressions for which clustering information is returned:

- For a table with no clustering key, this argument is required. If this argument is omitted, an error is returned.
- For a table with a clustering key, this argument is optional; if the argument is omitted, Snowflake uses the defined clustering key to return clustering information.

Even if only one column name or expression is passed, it must be inside parentheses.

Note

You can use this argument to return clustering information for any columns in the table, regardless of whether a clustering key is defined for the table.

In other words, you can use this to help you decide what clustering to use in the future.

SYSTEM\$CLUSTERING_INFORMATION - Output

cluster_by_keys - Columns in table used to return clustering information; can be any columns in the table.

notes - This column can contain suggestions to make clustering more efficient. For example, this field might contain a warning if the cardinality of the clustering column is extremely high.
This column can be empty.

total_paritition_count - Total number of micro-partitions that comprise the table.

total_constant_partition_count - Total number of micro-partitions for which the value of the specified columns have reached a constant state (i.e. the micro-partitions will not benefit significantly from reclustering). The number of constant micro-partitions in a table has an impact on pruning for queries. The higher the number, the more micro-partitions can be pruned from queries executed on the table, which has a corresponding impact on performance.

average_overlaps - Average number of overlapping micro-partitions for each micro-partition in the table. A high number indicates the table is not well-clustered.

average_depth - Average overlap depth of each micro-partition in the table. A high number indicates the table is not well-clustered.

partition_depth_histogram - A histogram depicting the distribution of overlap depth for each micro-partition in the table. The histogram contains buckets with widths:

- 0 to 16 with increments of 1.
- For buckets larger than 16, increments of twice the width of the previous bucket (e.g. 32, 64, 128, ...).

clustering_errors - An array of JSON objects, each with a timestamp and error name/value pair. The error describes why automatic clustering was not able to recluster data. By default, the 10 most recent errors are returned in the array. To return more or fewer errors, specify a number as the second argument of the function.

SYSTEM\$CLUSTERING_INFORMATION - Examples

```
SELECT SYSTEM$CLUSTERING_INFORMATION('test2', '(col1, col3)');
```

```
+-----+
| SYSTEM$CLUSTERING_INFORMATION('TEST2', '(COL1, COL3)') |
+-----+
| {                                                     |
|   "cluster_by_keys" : "LINEAR(COL1, COL3)",          |
|   "total_partition_count" : 1156,                   |
|   "total_constant_partition_count" : 0,             |
|   "average_overlaps" : 117.5484,                   |
|   "average_depth" : 64.0701,                       |
|   "partition_depth_histogram" : {                  |
|     "00000" : 0,                                    |
|     "00001" : 0,                                    |
|     "00002" : 3,                                    |
|     "00003" : 3,                                    |
|     "00004" : 4,                                    |
|     "00005" : 6,                                    |
|     "00006" : 3,                                    |
|     "00007" : 5,                                    |
|     "00008" : 10,                                   |
|     "00009" : 5,                                    |
|     "00010" : 7,                                    |
|     "00011" : 6,                                    |
|     "00012" : 8,                                    |
|     "00013" : 8,                                    |
|     "00014" : 9,                                    |
|     "00015" : 8,                                    |
|     "00016" : 6,                                    |
|     "00032" : 98,                                   |
|     "00064" : 269,                                  |
|     "00128" : 698,                                  |
|   },                                                 |
|   "clustering_errors" : [ {                         |
|     "timestamp" : "2023-04-03 17:50:42 +0000",      |
|     "error" : "(003325) Clustering service has been disabled.\n" |
|   }                                                 |
| }                                                     |
+-----+
```

SEARCH OPTIMIZATION SERVICE

The search optimization service aims to significantly improve the performance of selective point lookup queries on tables. A point lookup query returns only one or a small number of distinct rows.

Use case examples include:

- Business users who need fast response times for critical dashboards with highly selective filters.
- Data scientists who are exploring large data volumes and looking for specific subsets of data.

A user can register one or more tables to the search optimization service. Search optimization is a table-level property and applies to all columns with supported data types (see the list of supported data types further below).

HOW DOES SNOWFLAKE SEARCH OPTIMIZER SERVICE WORK

To **improve performance for point lookups**, the search optimization service relies on a persistent data structure that serves as an optimized search access path.

A **maintenance service** that runs in the background is responsible for creating and **maintaining the search access path**:

- When you add search optimization to a table, the maintenance service populates the data needed to perform the lookups.
- The process of populating data is done in the background
- When data in the table changes, the service automatically updates the search access path to reflect the changes to the data.
- Queries might run slower if the search access path hasn't updated but will always return up-to-date results.

QUERY PERFORMANCE OPTIMIZATION

Clustering: Can speed any of the following, as long as they are on the clustering key for **Range** and **Equality** searches. A table can be clustered on only a single key.

Search optimization service: speeds only **Equality** searches. However, this applies to all the columns of supported types in a table that has search optimization enabled.

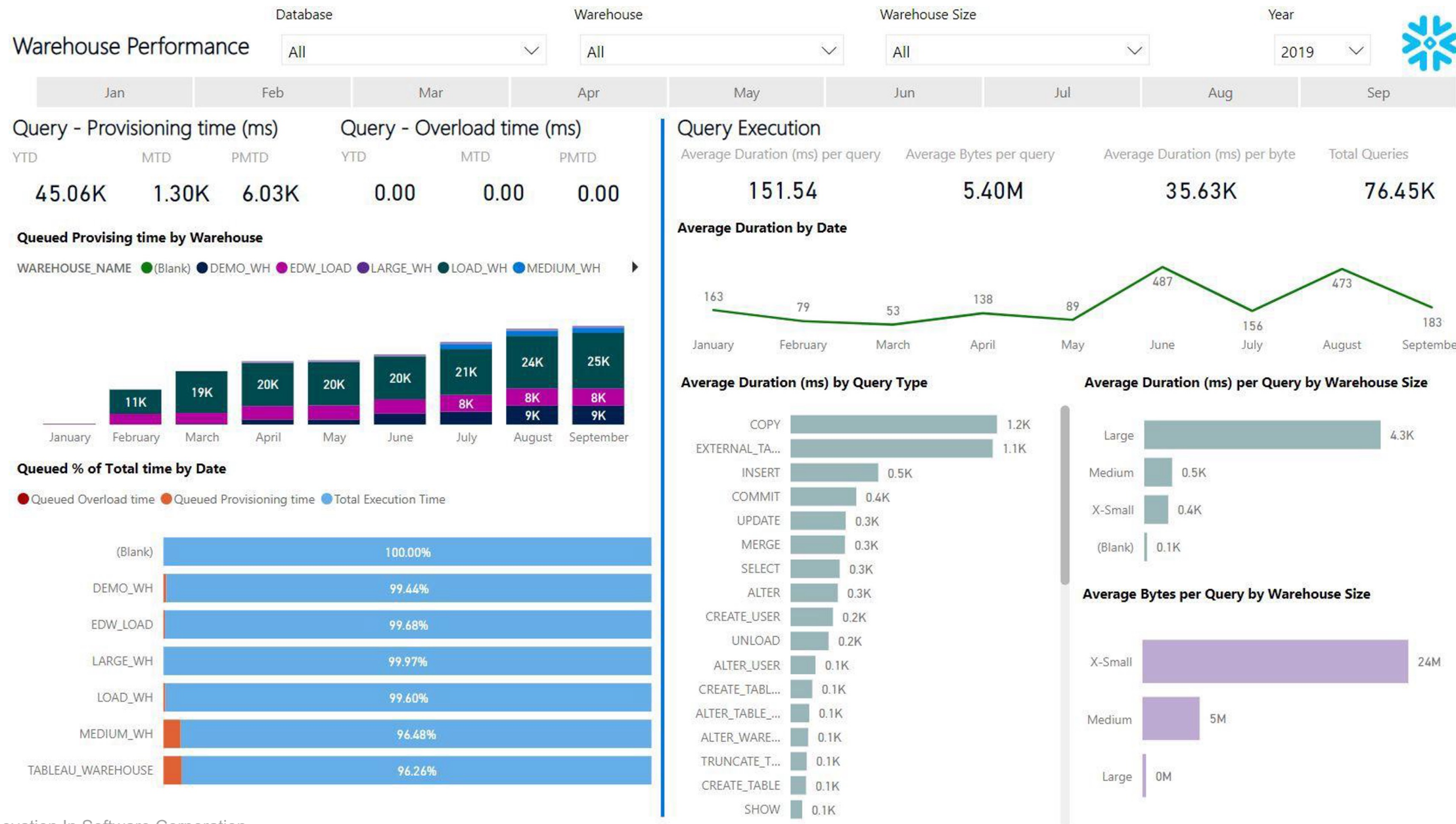
Materialized view: speeds both **Equality** searches and **Range** searches, as well as some sort operations, but only for the subset of rows and columns included in the materialized view

SCALING A VIRTUAL WAREHOUSE

Scaling up is all about increasing the compute power of the existing warehouse node. This should assist long-running queries, queries that require a lot of bytes scanned and queries with storage spillage. This would be done by increasing the size property of the warehouse.

Scaling Out is the process of adding more clusters to an existing warehouse. This will assist when there are a large number of concurrent queries being executed in the same warehouse. Scaling Out will allow for those queued queries to be executed on the new provisioned cluster.

WAREHOUSE PERFORMANCE REPORT



WAREHOUSE REPORT KEY METRICS

Queued Overload Time - Time (in milliseconds) spent in the warehouse queue, due to the warehouse being overloaded by the current query workload.

Check the Autoscaling Policy - Check the number of clusters is large enough for the number of concurrent queries

Queued Provisioning time - Time (in milliseconds) spent in the warehouse queue, waiting for the warehouse servers to provision, due to warehouse creation, resume, or resize.

Query Type - DML, query, etc. If the query is currently running, or the query failed, then the query type may be UNKNOWN. This will show the type of query that is running (e.g. Select, Copy Into, Alter).

Execution Time - Execution time (in milliseconds)

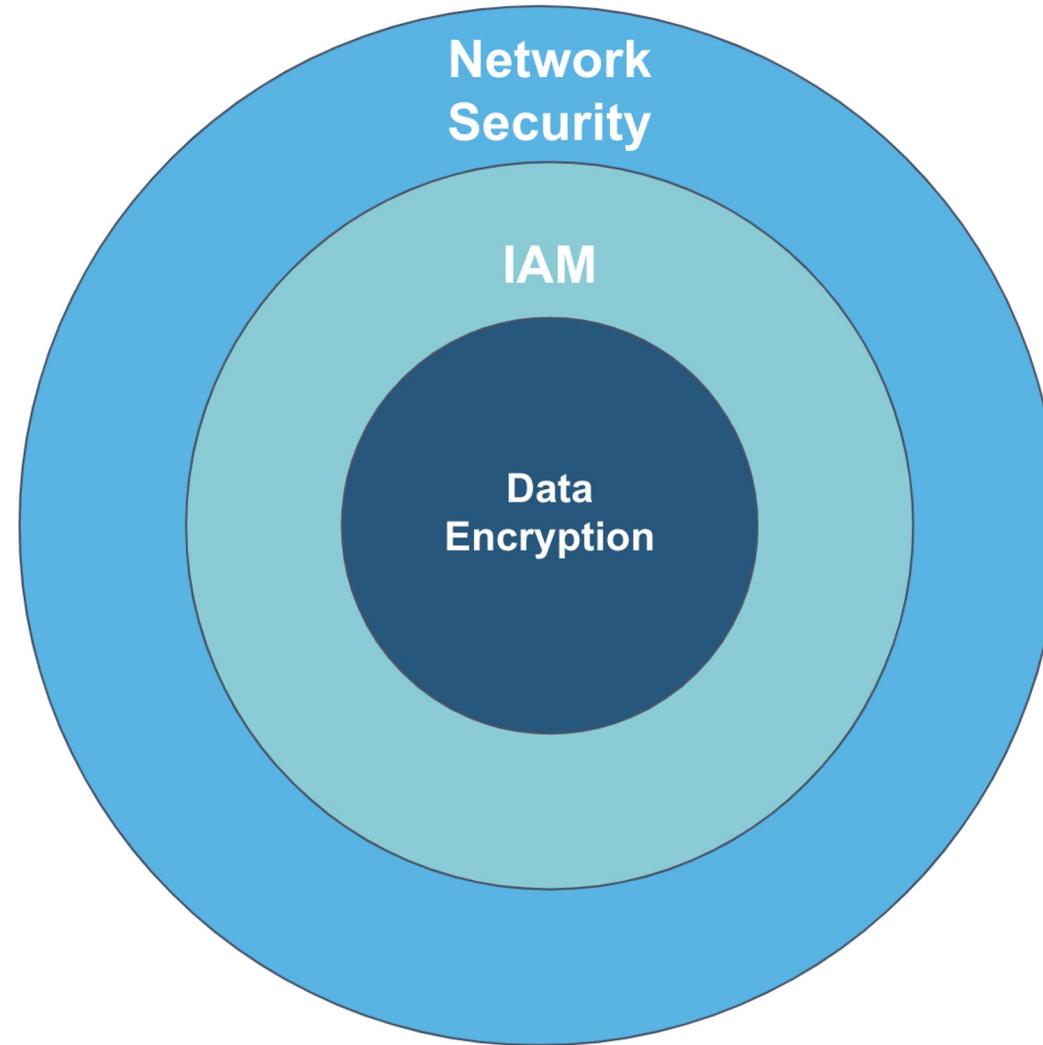
Bytes Scanned - Number of bytes scanned by this statement. The time taken and bytes scanned may indicate a larger warehouse is required.

Managing Security

In Snowflake

6

DEFENSE IN DEPTH



DATA ENCRYPTION

Encrypting data means applying an encryption algorithm to translate the clear text into cipher text.

Encrypt data:

- from the time it leaves your premises
- through the Internet
- into the cloud data warehouse
- stored on disk
- moved into a staging location
- placed within a database object
- cached within a virtual data warehouse.

AUTHENTICATION

Any data service or data warehouse should always authorize users, authenticate credentials, and grant users access only to the data they're authorized to access.

Data breaches often result from users selecting weak passwords coupled with rudimentary authentication procedures.

IDENTITY VS ACCESS

Identity – who you are

Access – what you're allowed to do

Authentication – proving your identity, e.g user
SUSANMARTIN with password Sn0wFl@ke

Authorization – the right to do something,
SUSANMARTIN – ACCOUNTADMIN aka Snowflake
superuser

AUTHORIZATION AND RBAC

Snowflake authorization allows you to do things in the data warehouse **based on your role**, that's called **Role-Based Access Control (RBAC)**.

ROLE SWITCHING

Snowflake allows you to **switch roles** without doing further authentication.

The standard roles are

ACCOUNTADMIN

SYSADMIN

SECURITYADMIN

USERADMIN

PUBLIC

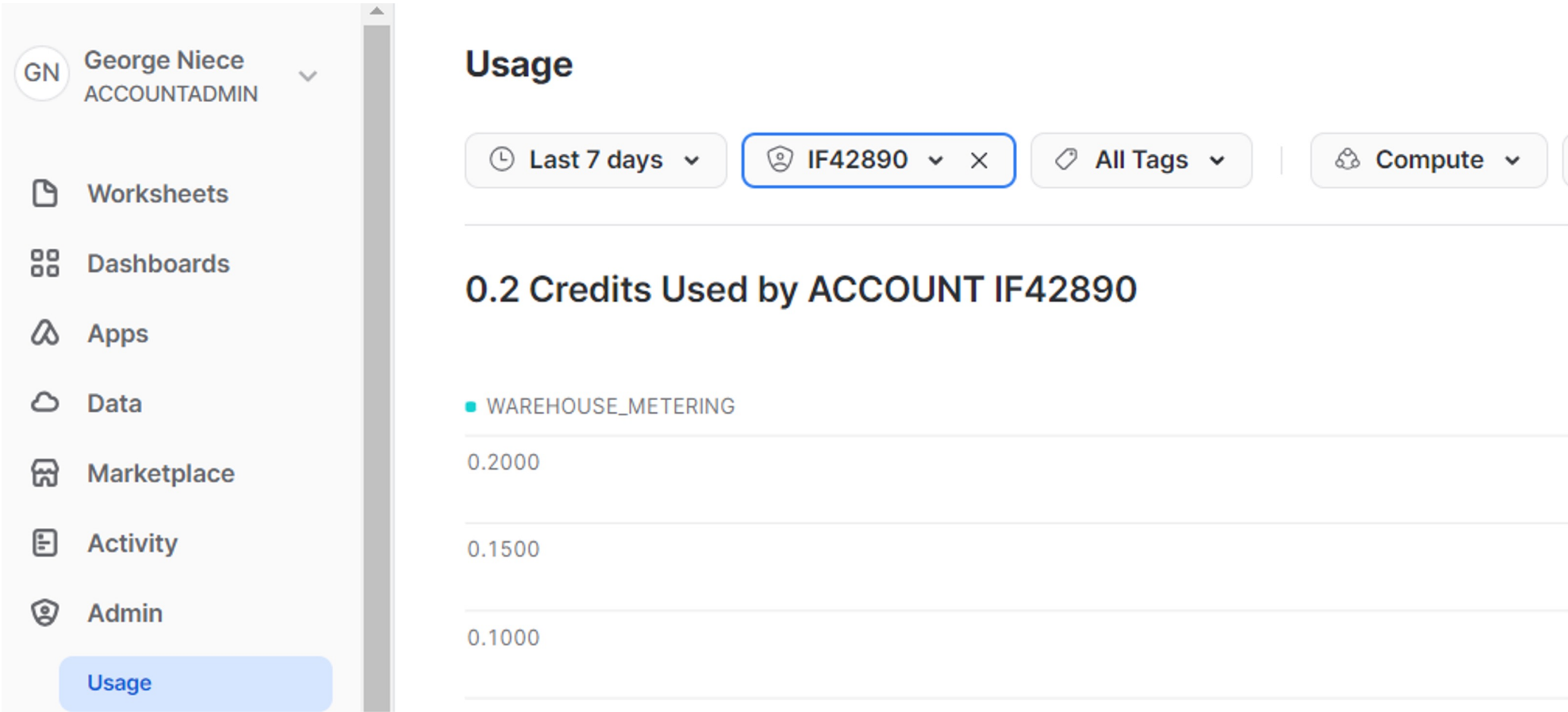
PUBLIC DATABASE ACCESS

Snowflake provides functions to view role availability. For example, to see what roles you have available you could execute

```
select current_available_roles();
```

USERS & ROLES

ACCOUNTADMIN is the only role that has default access to the Account Usage in the Snowflake Console



MANAGING USERS AND ROLES

Snowflake recommends using SCIM where supported by your Identity Provider to provision and externally manage users and roles in Snowflake.

Identity Providers can be further configured to synchronize users and roles with your Active Directory users and groups. Examples include Okta SCIM and Azure AD SCIM



SAML OR OAUTH

SAML and OAuth are great protocols within identity management. SAML is based around user authentication and OAuth is based around authorization.

If you're using an external identity provider such as Okta, it's likely you have the ability to use both SAML and OAuth for a variety of applications and integrations. Specifically with regards to Okta, it would be recommended to use OAuth because the specification has more levels of trust and security, and you can configure different authorization flows for each one of these.

In either case, this is an area that requires experience and Subject Matter Expertise (SME) to most correctly secure.

ACCESS CONTROL FRAMEWORK

Snowflake's approach to access control combines aspects from both of the following models:

- **Discretionary Access Control (DAC):** Each object has an owner, who can in turn grant access to that object.
- **Role-based Access Control (RBAC):** Access privileges are assigned to roles, which are in turn assigned to users.

ACCESS CONTROL KEY CONCEPTS

Securable object: An entity to which access can be granted. Unless allowed by a grant, access will be denied.

Role: An entity to which privileges can be granted. Roles are in turn assigned to users. Note that roles can also be assigned to other roles, creating a role hierarchy.

Privilege: A defined level of access to an object. Multiple distinct privileges may be used to control the granularity of access granted.

User: A user identity recognized by Snowflake, whether associated with a person or program.

SNOWFLAKE AUTHENTICATION METHODS

Snowflake Drivers, CLI

Password

OAuth

Key Pair

External Browser

Okta native auth

Snowflake UI

Password

SAML

Snowpipe

Key Pair

AUTHENTICATION BEST PRACTICE

Preference #1: OAuth (either Snowflake OAuth or External OAuth)

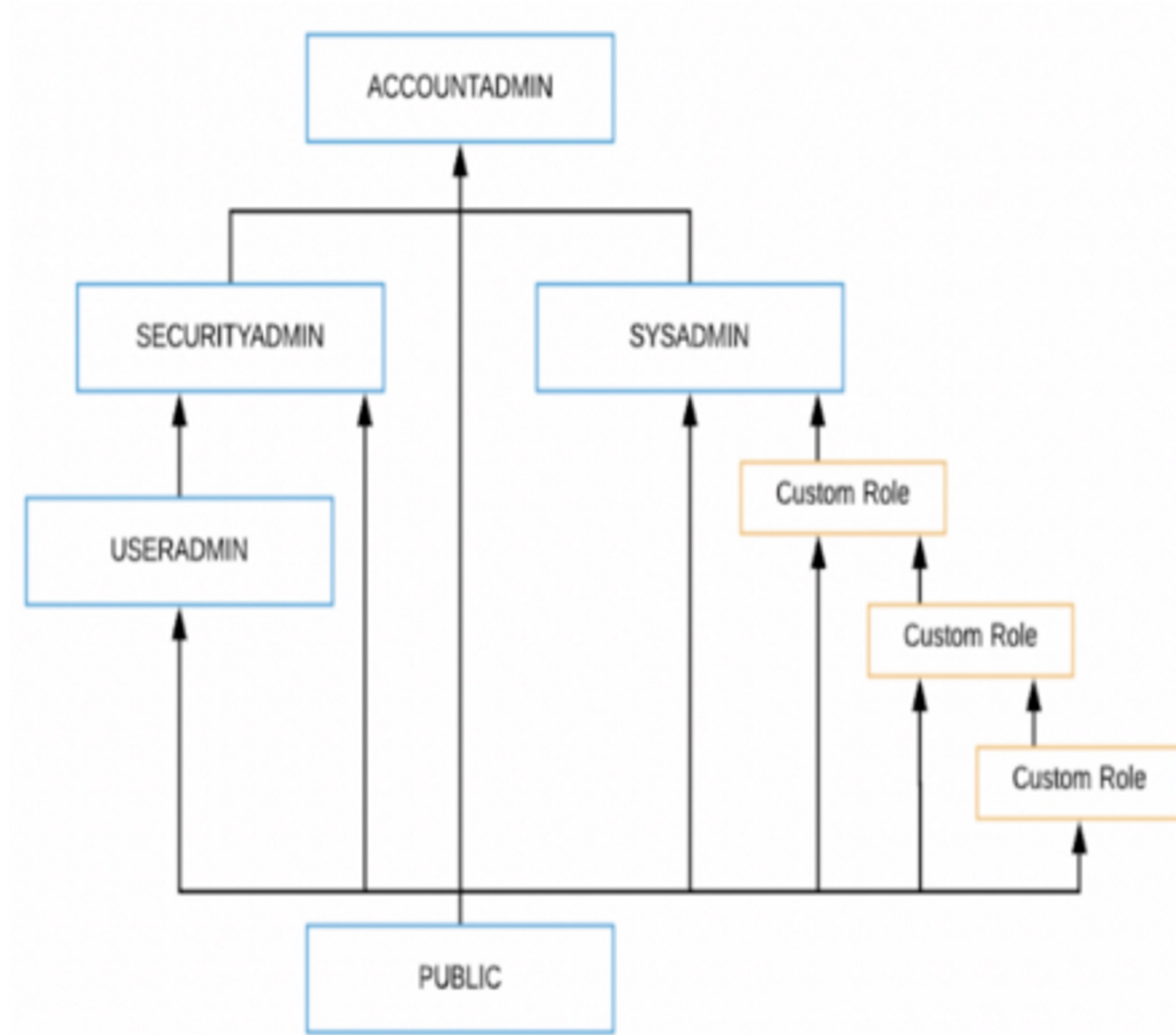
Preference #2: External Browser, if it's a desktop application that doesn't support OAuth

Preference #3: Okta native authentication, if you're using Okta, and the app supports this method while not supporting OAuth or external browser authentication yet.

Preference #4: Key Pair Authentication, mostly used for service account users. Since this requires the client application to manage private keys, complement it with your internal key management software.

Preference #5: Password, this should be the last option for applications that don't support any of the above options. This option is commonly used for service account users connecting from 3rd party ETL apps.

ROLE-BASED ACCESS CONTROL (RBAC)



LOGICAL RBAC LEVELS

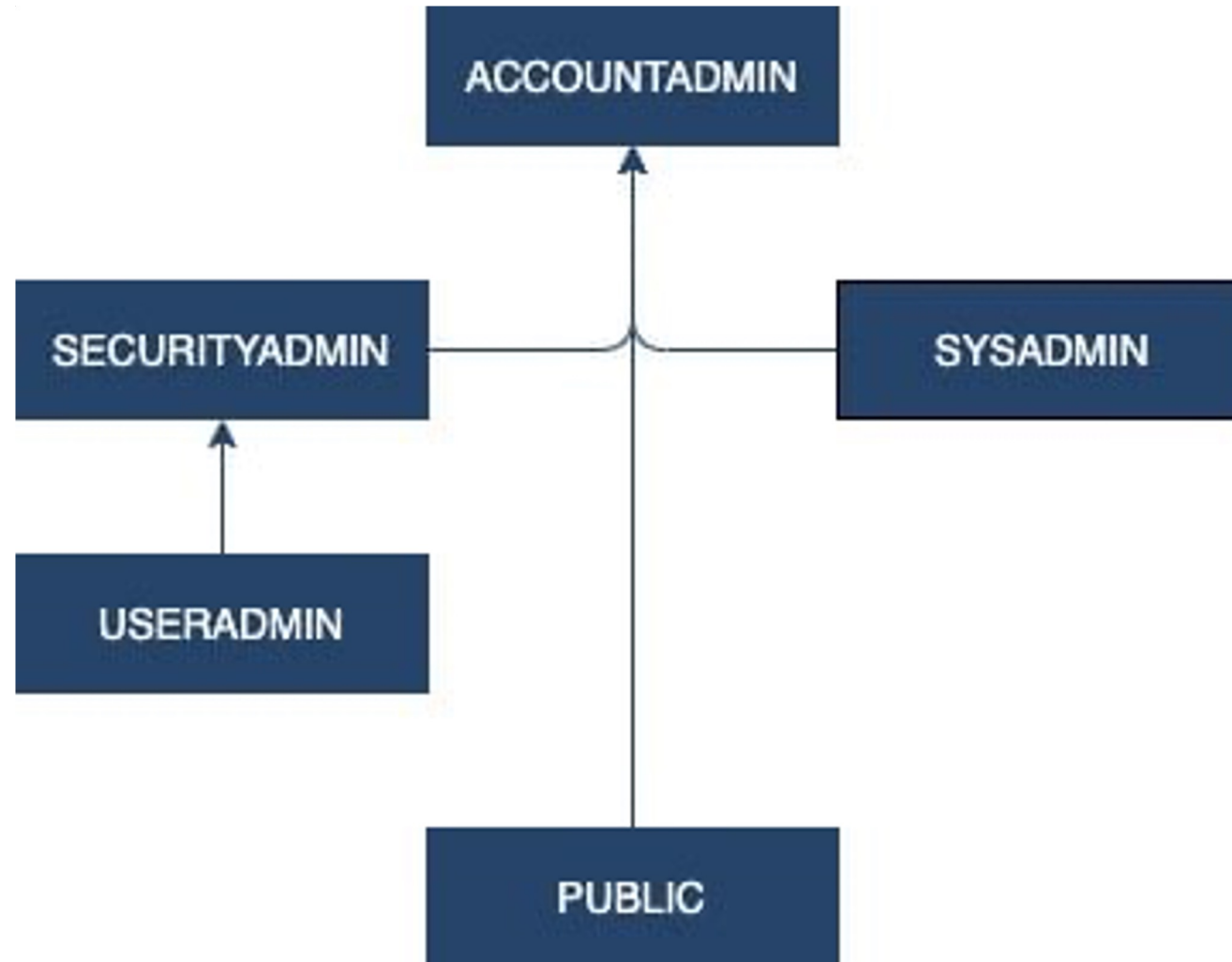
Access Roles: These roles will be the lowest level which will have actual access privileges on DB objects. Maintain distinct access roles based on user requirements at a schema level.

Functional Roles: These roles map to the actual real-world roles of the users in the organization and are assigned to the Snowflake users.

Domain Roles: If the organization has a requirement to have multiple independent domains under the same account, this will help to realize that. E.g. when there needs to be a separation of production or UAT/development domains.

System Roles: Native Snowflake system role to which all the Domain roles should be rolling up to.

SYSTEM-DEFINED ROLES



SYSTEM-DEFINED ROLES

ACCOUNTADMIN

This is the highest level role in Snowflake. It should be heavily limited in access and encapsulates SYSADMIN and SECURITYADMIN

SECURITYADMIN

This role is used for managing any object grant globally, and therefore is granted the MANAGE GRANTS privilege by default. This role also inherits the USERADMIN role.

USERADMIN

This role is dedicated to user/role management. By default, this role can create/modify users and their respective roles (assuming those roles/users haven't been transferred to another role).

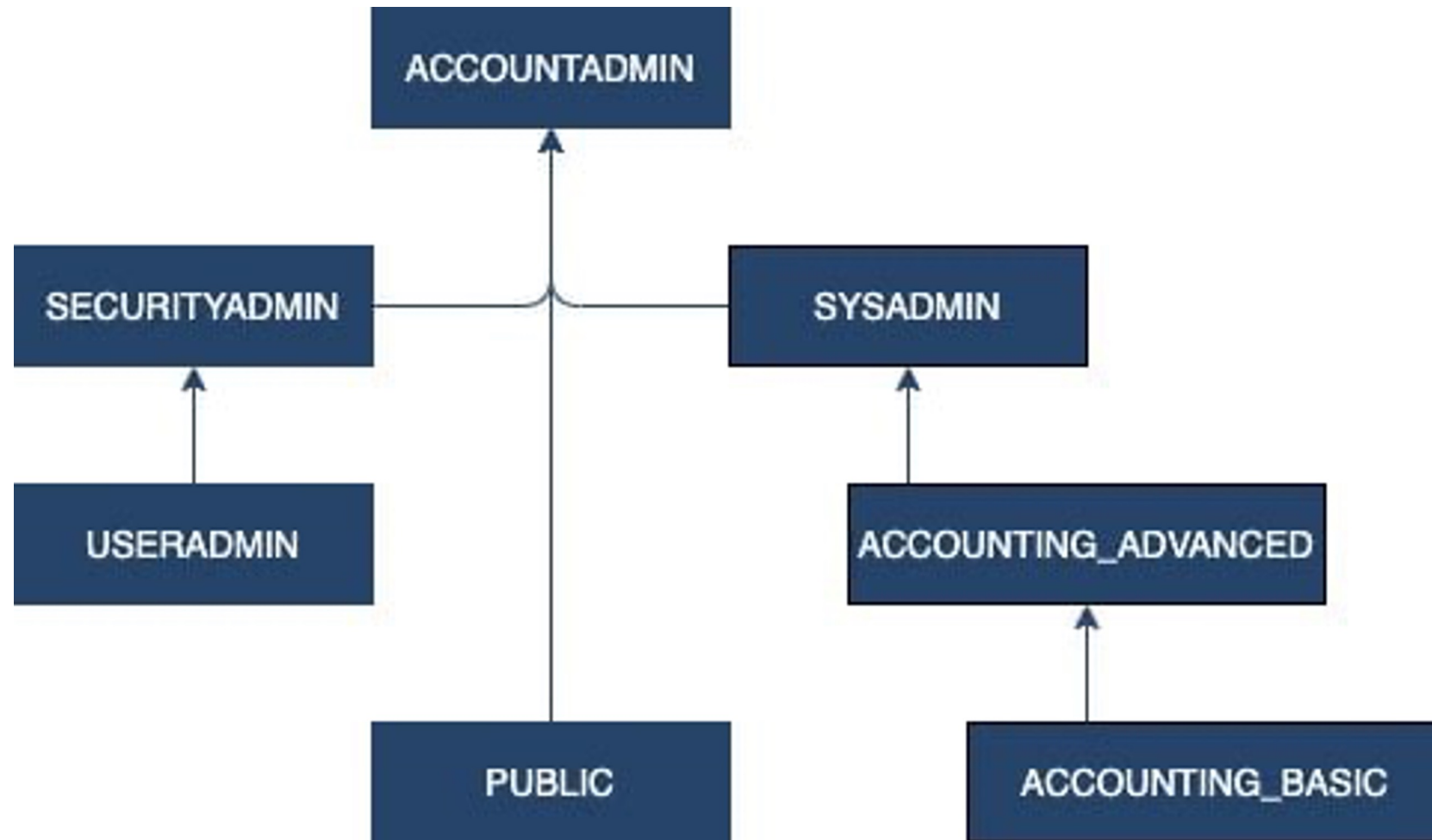
SYSADMIN

This role is dedicated to system object management. It is recommended by Snowflake to assign all custom roles to the SYSADMIN role, so this role can grant these privileges to other roles.

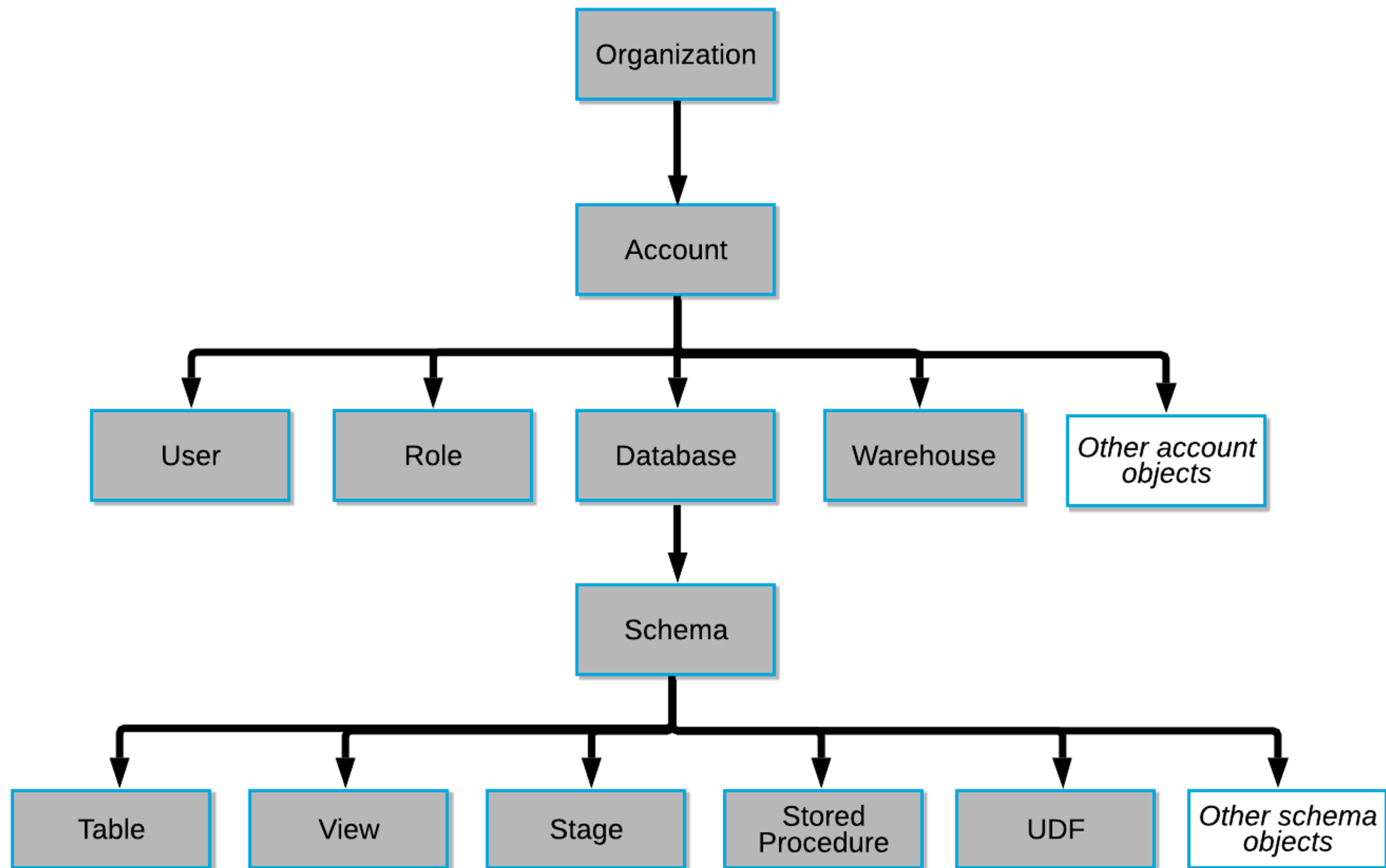
PUBLIC

This is a pseudo role that every user/role in the account gets. Things owned by the public role are, well, public!

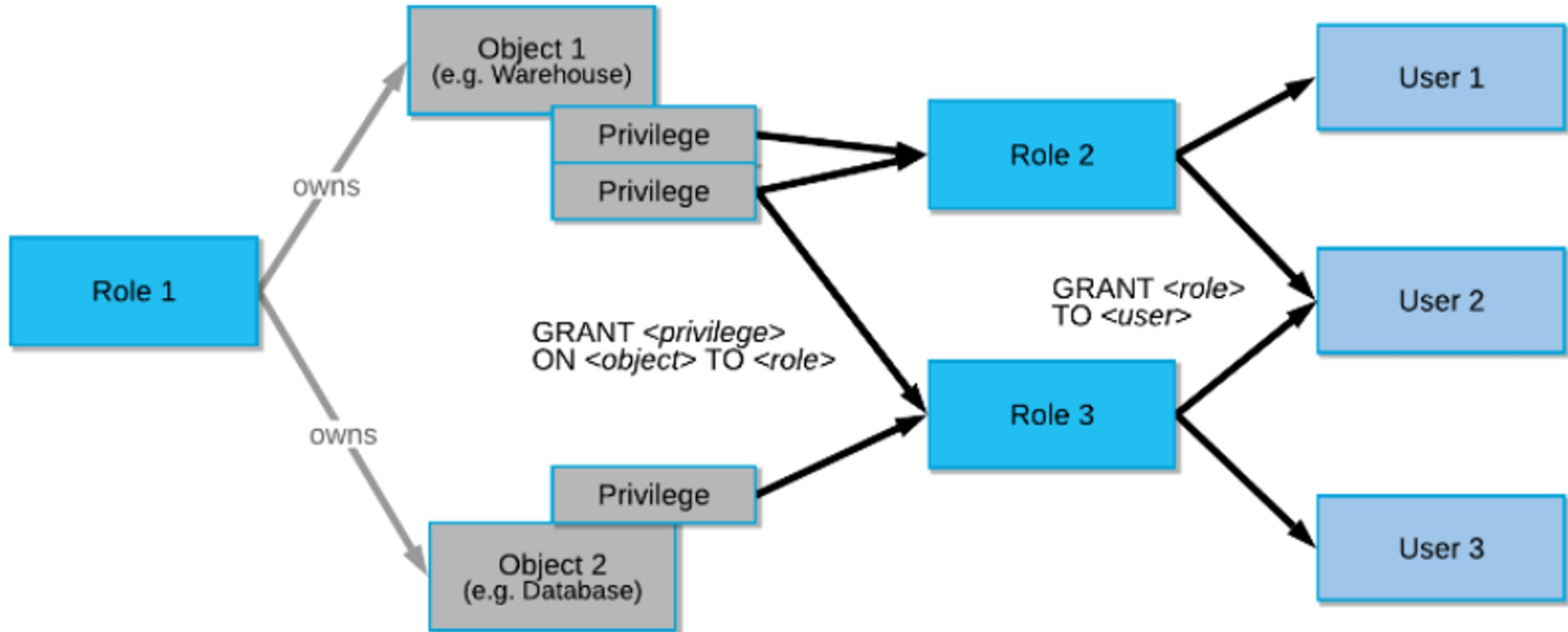
CUSTOM ROLES



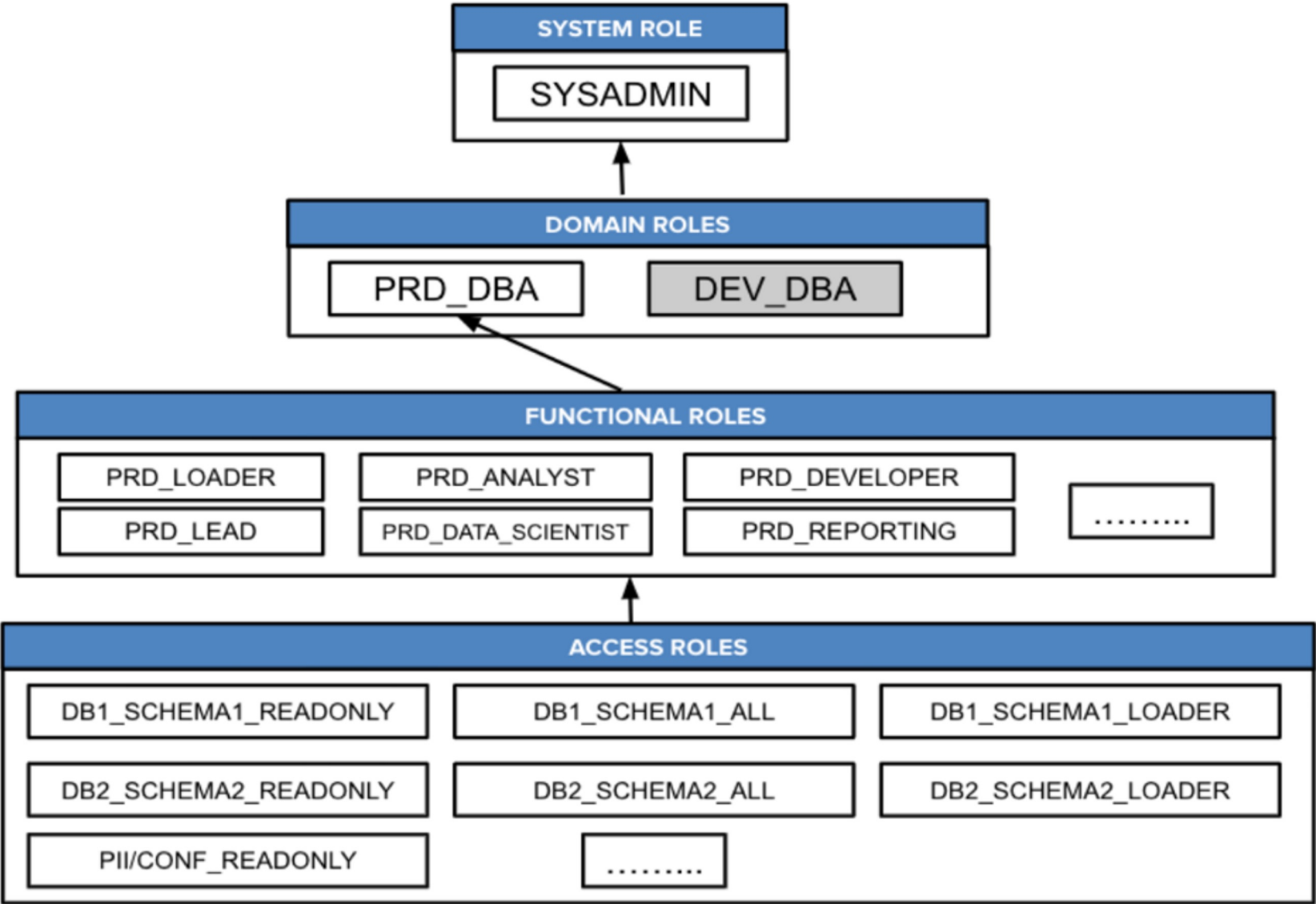
SECURABLE OBJECTS



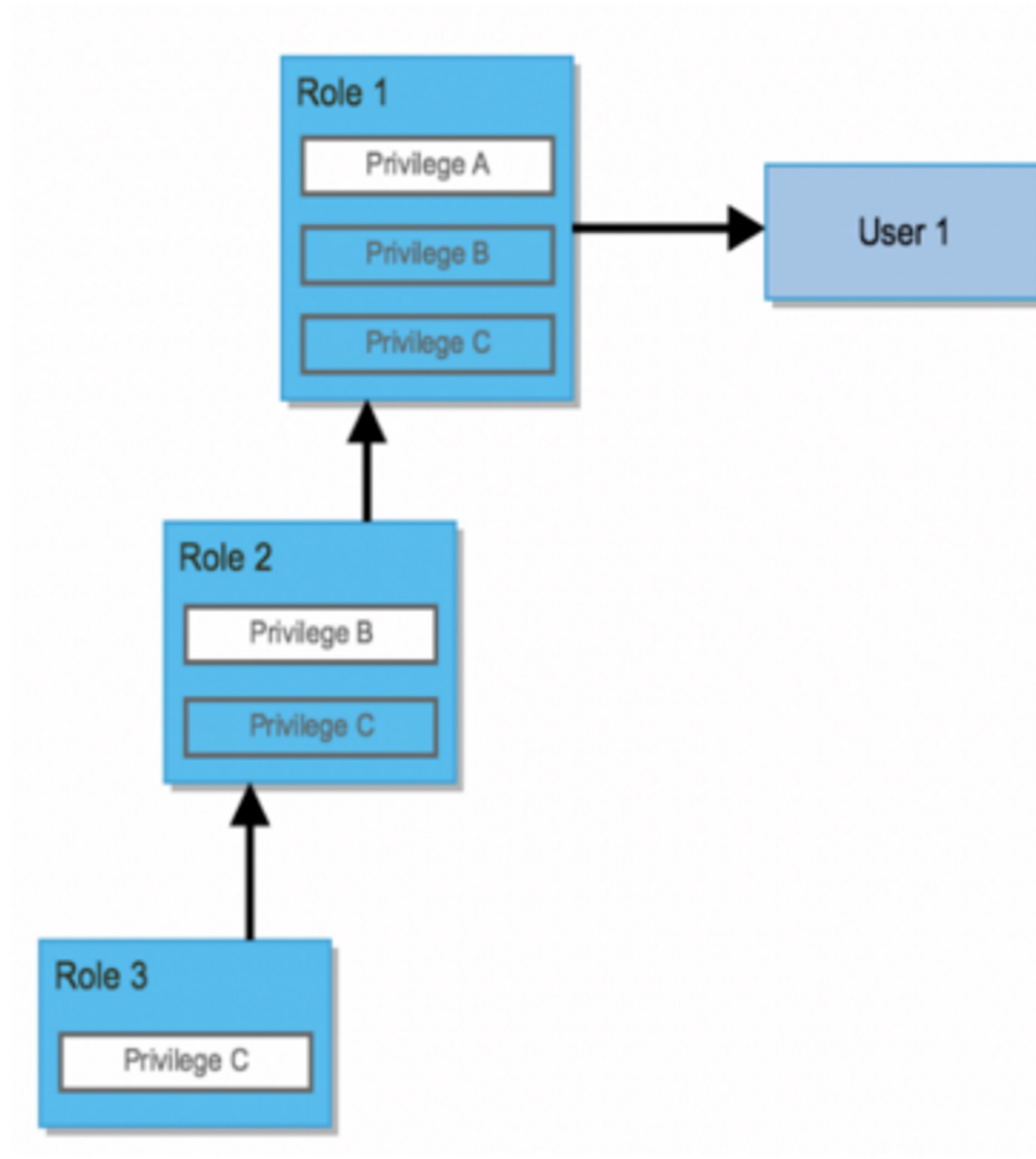
PRIVILEGES



RBAC ROLE HIERARCHY



PRIVILEGE INHERITANCE

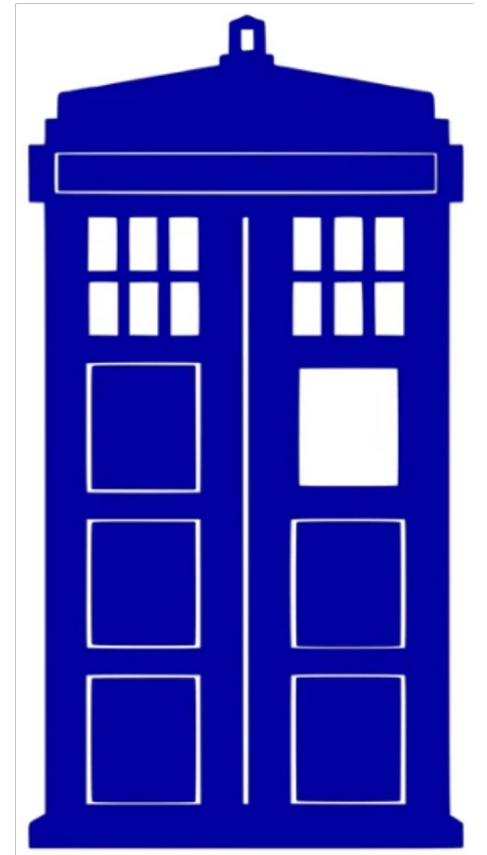


TIME TRAVEL

Snowflake provides powerful CDP features for ensuring the maintenance and availability of your historical data (i.e. data that has been changed or deleted):

Querying, cloning, and restoring historical data in tables, schemas, and databases for up to 90 days through Snowflake Time Travel.

These features are included standard for all accounts, i.e. no additional licensing is required; however, standard Time Travel is 1 day. Extended Time Travel (up to 90 days) requires Snowflake Enterprise Edition.

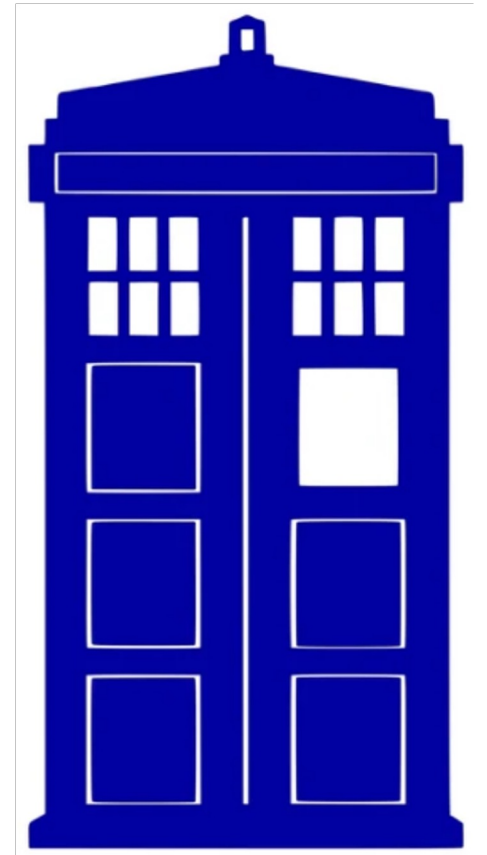


TIME TRAVEL IN ACTION

You accidentally DROP a table in production

In other RDBMS and data warehouse systems you start to work on a restore.

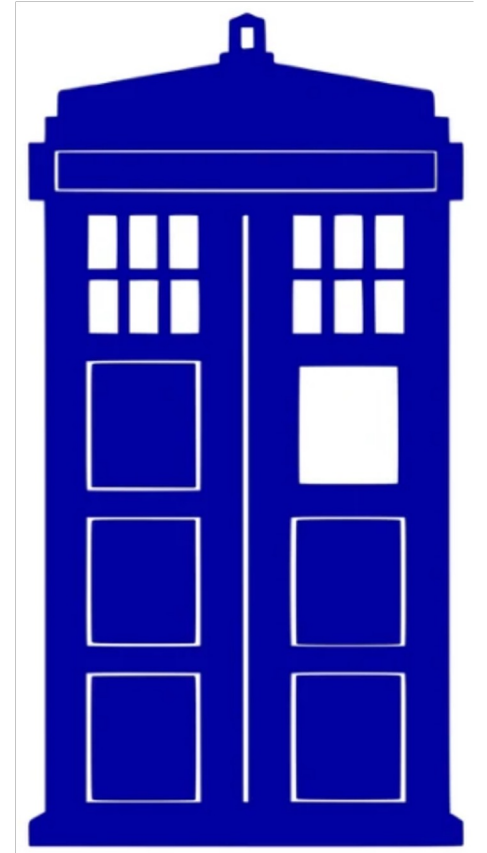
In Snowflake we turn to the UNDROP command



SNOWFLAKE UNDROP

UNDROP TABLE *TABLE_NAME*

- Restoring tables is only supported in the current schema or current database, even if the table name is fully-qualified.
- If a table with the same name already exists, an error is returned.



TIME TRAVEL AT | BEFORE

The AT or BEFORE clause is used for Snowflake Time Travel. In a query, it is specified in the FROM clause immediately after the table name and it determines the point in the past from which historical data is requested for the object:

The AT keyword specifies that the request is inclusive of any changes made by a statement or transaction with timestamp equal to the specified parameter.

The BEFORE keyword specifies that the request refers to a point immediately preceding the specified parameter.

TIME TRAVEL AT | BEFORE

```
set query_id = (select query_id from  
table(information_schema.query_history_by_session  
(result_limit=>5)) where query_text like 'update%' order by start_time  
limit 1
```

```
create or replace table trips as  
(select * from trips before (statement => $query_id));
```

TIME TRAVEL LIMITATIONS

Databases, schemas, and tables can be recovered by Time Travel.

With tables, Time Travel support depends on the type of table:

- Only **permanent** can be recovered longer than 1 day ago.
- **Temporary** and **transient** tables can leverage Time Travel for up to one day.
- **External** tables can't use Time Travel, as they are not in Snowflake.

FAIL-SAFE

Separate and distinct from Time Travel, Fail-safe ensures historical data is protected in the event of a system failure or other catastrophic event, e.g. a hardware failure or security breach.

Disaster recovery of historical data (by Snowflake) through Snowflake Fail-safe.

Both Time Travel and Fail-safe require additional data storage, which has associated fees.



FAIL-SAFE DETAILS

Fail-safe is ***not*** provided as a means for accessing historical data after the Time Travel retention period has ended.

It is for use ***only*** by Snowflake to recover data that may have been lost or damaged due to extreme operational failures.

Data recovery through Fail-safe may take from several hours to several days to complete.



FAIL-SAFE LIMITATIONS

Databases, schemas, and tables can be recovered by Fail-Safe.

When it comes to tables in Snowflake, we have the following rules:

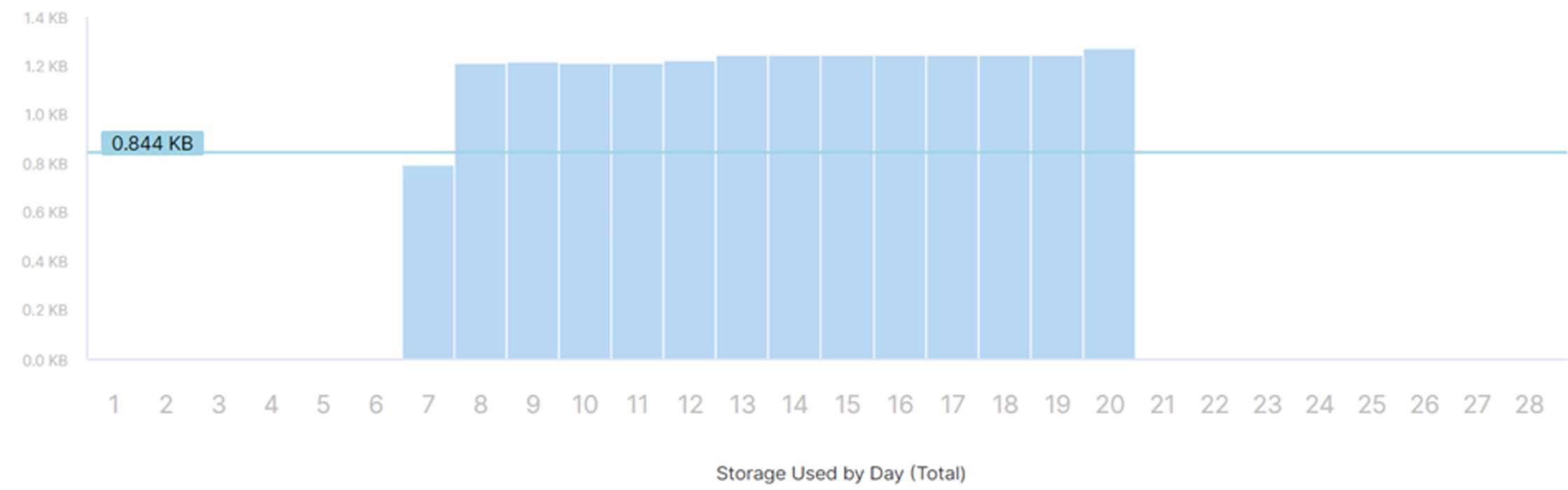
- Permanent tables can leverage Fail-Safe.
- Temporary and transient tables are not able to setup Fail-Safe.
- External tables can't use Fail-Safe as the data is not stored in Snowflake.

ACCOUNT – USAGE - FAIL-SAFE

LBA64704 - Usage

November 20

Warehouses	Credits Used	Average Storage Used	Data Transfer	Total	Database	Stage	Fail Safe
2	1.00	0.844 KB	0 B				



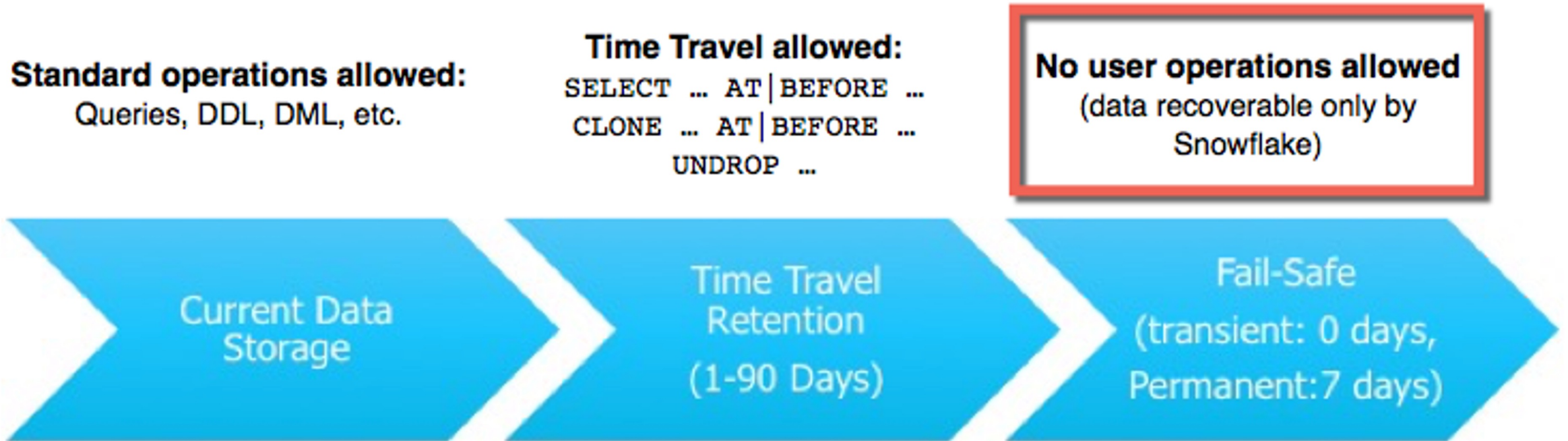
FAIL-SAFE IN ACTION

At high level, Fail-safe provides a (non-configurable) 7-day period during which historical data is recoverable by Snowflake support team.

You will be billed for storage for & days for your databases and tables in Fail-Safe.



CONTINUOUS DATA PROTECTION LIFECYCLE



Core Governance Features

- **Snowflake's Governance Features**
 - **Column level Security:** Mask specific columns
 - **Row level Security:** control visible rows in queries
 - **Object Tagging:** Track sensitive data attributes
 - **Tag-based Masking:** Linking Masking to data tags
 - **Data Classification:** Categorize data for compliance

Audit & Monitoring

- **Tracking & Auditing in Snowflake**
 - **Access History:** Review user access via ACCESS_HISTORY View
 - **Object Dependencies:** Understand object references via OBJECT_DEPENDENCIES view
 - **Snowsight Governance:** Monitor policy & tag usage with the Data > Governance area

Snowflake Object Tagging

- **Definition:** Tags are schema-level objects that can be attached to Snowflake objects for monitoring, compliance, discovery, and resource usage.
- **Usage:**
 - Centralized or decentralized data governance
 - Allows for monitoring sensitive data and resource usage efficiently
- **Creation:**
 - Use a `CREATE TAG` statement to define a tag
 - Assign a tag when creating an object or modify an existing one using `ALTER <object> statement`

Assigning and Using Tags

- **Tag Attributes:**

- Consist of a key-value pair; the key is unique per schema
- String value can be either duplicated or unique among objects

- **Applications:**

- Discovering database objects with sensitive information
- Making data available using row access policies or masking policies
- Monitoring resource usage by querying tags, e.g., for cost centers or projects

Tag Quotas and Management

Quotas:

- Tag string value can be up to 256 characters
- 50 unique tags maximum per object, but 100 tags can be specified in a single statement
- Tables/views and their columns have separate quotas

Management:

- Manage tags using ALTER and DROP TAG statements
- Dropped tags remain in system for 24 hours to allow potential recovery

Tag Lineage and Inheritance

Definition: Tags can be inherited based on the Snowflake securable object hierarchy.

Inheritance:

- Tags on a table apply to its columns (tag lineage)
- Overriding is possible for a more specific value or to add a new tag

Nested Objects:

- Tags don't automatically propagate to nested objects
- Explicitly set tags for nested objects using CREATE OR REPLACE statement if necessary

Object Dependencies in Snowflake

- **Definition:** Object dependencies exist when an object references metadata for itself or another object.
- **Location:** Tracked in the Account Usage view `OBJECT_DEPENDENCIES`.
- **Applications:**
 - Support in local Snowflake accounts.
 - Dependencies related to data sharing (e.g., consumer views from provider shares).
- **Benefits:**
 - Ensure data integrity.
 - Compliance with regulatory standards.
 - Detailed impact analysis.

Types of Dependencies

- **BY_NAME:**
 - Based on the object's name value.
 - Example: A view (myview) referencing a table (mytable) using its name.
- **BY_ID:**
 - Based on the object's ID value.
 - Example: An external stage (my_ext_stage) storing the OBJECT_ID of a storage integration (myint).
- **BY_NAME_AND_ID:**
 - Depends on both the object's name and ID values.
 - Common in objects like materialized views due to statements like CREATE OR REPLACE.

Benefits of Identifying Object Dependencies

- Impact Analysis:
 - Understand relationships between referencing and referenced objects.
 - Ensure updates don't negatively impact users.
 - Example: Coordinating table and view updates to avoid breaking queries.
- Compliance:
 - Identify relationships between sensitive data sources and targets.
 - Address compliance requirements like GDPR effectively.
- Data Integrity:
 - Gives confidence to data professionals about the trustworthiness of data sources.

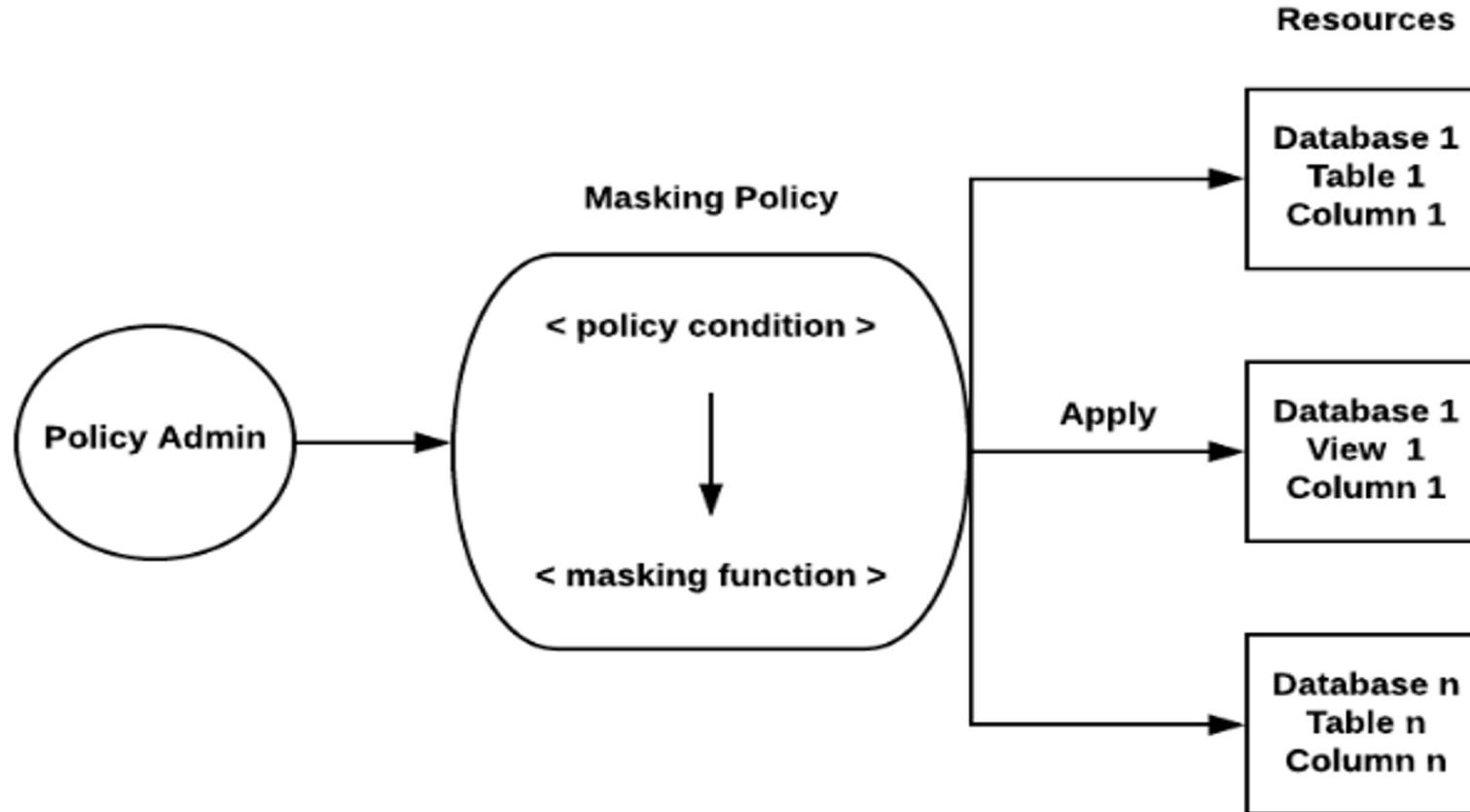
Column Level Security in Snowflake

- **Overview of Column-level Security**
 - Purpose: Applies a masking policy to a column within a table or view.
 - Features: Includes two main features - Dynamic Data Masking and External Tokenization.
- **Dynamic Data Masking**
 - Function: Uses masking policies to selectively mask plain-text data in table and view columns at query time.
 - Impact: Ensures sensitive data is not exposed to unauthorized users during query execution.
- **External Tokenization**
 - Process: Enables tokenizing data before loading it into Snowflake and detokenizing at query runtime.
 - Tokenization: Replacing sensitive data with an undecipherable token.
 - Implementation: Utilizes masking policies with external functions for the tokenization process.
- **Significance of Column-level Security**
 - Data Protection: Provides robust protection of sensitive data within Snowflake.
 - Flexibility: Offers different methods to secure data according to organizational needs and compliance requirements.
 - Dynamic Application: Both features are applied dynamically at query time, ensuring data security without altering the stored data.
- **Key Takeaway:**
 - Enhanced Data Security: Column-level Security in Snowflake, through Dynamic Data Masking and External Tokenization, provides a powerful mechanism to secure and mask sensitive data, ensuring only authorized access at the column level.

Understanding Masking Policies in Snowflake

- **Role of Masking Policies**
 - Purpose: Protect sensitive data from unauthorized access while allowing authorized users to see the data at query runtime.
 - Operation: Applied dynamically; sensitive data in tables isn't modified (no static masking).
- **How Masking Policies Work**
 - Query Execution: During a query, the policy conditions determine if unauthorized users see masked, partially masked, obfuscated, or tokenized data.
 - Transformation: Can include conditions and functions to transform data at query runtime based on those conditions
- **Flexibility and Management**
 - Schema-Level Object: Masking policies are schema-level objects, allowing for centralized, decentralized, or hybrid management approaches.
- **Segregation of Duties**
 - Policy-Driven Approach: Supports segregation of duties, allowing security teams to limit sensitive data exposure.
 - Scope: Can limit exposure even to object owners (roles with OWNERSHIP privilege) who normally have full access.
- **Key Takeaway:**
 - Strategic Data Protection: Masking policies in Snowflake provide a strategic, dynamic, and flexible approach to protect sensitive data at query runtime, supporting various management approaches and ensuring data privacy across different levels of access.

Masking Policies



Masking Policies

Authorized role (i.e. SUPPORT)

ID	Phone	SSN
101	408-123-5534	387-78-3456
102	510-334-3564	226-44-8908
103	214-553-9787	359-9987-0098

Unauthorized role (i.e. ANALYST)

ID	Phone	SSN
101	***-**-5534	*****
102	***-**-3564	*****
103	***-**-9787	*****

Components and Advantages of Masking Policies in Snowflake

Components of a Masking Policy

- Data Type: Single data type per policy.
- Conditions: One or more conditions, expressed using Conditional Expression Functions, Context Functions, or querying a custom entitlement table.
- Masking Functions: Built-in functions (e.g., REGEXP_REPLACE, SHA2, SHA2_HEX), User-Defined Functions, or External Functions (for de-tokenization).

Application of Masking Policies

- Scope: Can be applied to one or more table/view columns with the matching data type.
- Example: Define a policy for an email address and apply it to thousands of email columns across databases and schemas.

Advantages Over Secure Views

- Management Ease: Avoids the proliferation of views and BI dashboards, simplifying management.
- Segregation of Duties (SoD): Supports SoD through role separation of policy administrators and object owners.
- Limitations of Secure Views: Secure views do not support SoD, limiting their utility.

Default Settings and Privileges

- Object Owners: Those with OWNERSHIP privilege do not have the privilege to unset masking policies.
- Data Visibility: Object owners cannot view column data where a masking policy applies.

Key Takeaway:

- Enhanced Data Security and Management: Masking policies in Snowflake offer a streamlined, secure, and flexible approach to data protection, overcoming the limitations of secure views and supporting effective data governance and compliance.

Snowflake Practice Questions



2

POP QUIZ:

Mock Exam Questions



A Data Engineer is preparing to load staged data from an external stage using a task object

Which of the following practices will provide the MOST efficient load performance?

- a) Store the files on the external stage to ensure caching is maintained
- b) PUT all files in a single directory
- c) Limit file names to under 30 characters
- d) Organize files into logical paths that reflect a scheduling pattern



POP QUIZ:

Mock Exam Questions



A Data Engineer is preparing to load staged data from an external stage using a task object

Which of the following practices will provide the MOST efficient load performance?

- a) Store the files on the external stage to ensure caching is maintained
- b) PUT all files in a single directory
- c) Limit file names to under 30 characters
- d) Organize files into logical paths that reflect a scheduling pattern



POP QUIZ:

Mock Exam Questions



A Data Engineer is working on a project that requires data to be moved directly from an internal stage to an external stage

Which of the following is the QUICKEST way to accomplish this

- a) `COPY INTO @myExtStage from (Select $1, $2, ... @myInternalStage);`
- b) Copy the data from the internal stage to a table and then unload the data to an external stage
- c) `COPY INTO @myExtStage from @myInternalStage;`
- d) Write a custom script to move the data



POP QUIZ:

Mock Exam Questions



A Data Engineer is working on a project that requires data to be moved directly from an internal stage to an external stage

Which of the following is the QUICKEST way to accomplish this

- a) `COPY INTO @myExtStage from (Select $1, $2, ... @myInternalStage);`
- b) Copy the data from the internal stage to a table and then unload the data to an external stage
- c) `COPY INTO @myExtStage from @myInternalStage;`
- d) Write a custom script to move the data



POP QUIZ:

Mock Exam Questions



The s1 schema contains two permanent tables that were created as shown below:

```
CREATE TABLE table_a (c1 INT)  
DATA_RETENTION_TIME_IN_DAYS = 10;  
CREATE TABLE table_b (c1 INT);
```

What will be the impact of running the following command?

```
ALTER SCHEMA S1 SET  
DATA_RETENTION_TIME_IN_DAYS = 20;
```

- a) The retention time on table_a does not change; table_b is set to 20 days.
- b) An error will be generated, a data retention time on a schema cannot be set
- c) The retention time on both tables will be set to 20 days
- d) The retention time will not change on either table

POP QUIZ:

Mock Exam Questions



The s1 schema contains two permanent tables that were created as shown below:

```
CREATE TABLE table_a (c1 INT)
DATA_RETENTION_TIME_IN_DAYS = 10;
CREATE TABLE table_b (c1 INT);
```

What will be the impact of running the following command?

```
ALTER SCHEMA S1 SET
DATA_RETENTION_TIME_IN_DAYS = 20;
```

- a) The retention time on table_a does not change; table_b is set to 20 days.
- b) An error will be generated, a data retention time on a schema cannot be set
- c) The retention time on both tables will be set to 20 days
- d) The retention time will not change on either table

POP QUIZ:

Mock Exam Questions



What are the steps to load data from an archival cloud storage class (for example, Amazon S3 Glacier or Microsoft Azure Archive Storage) into Snowflake?

- A. Same steps as loading data from a non-archival cloud storage class.
- B. You should create an ARCHIVAL stage; then, you can perform the same steps as loading data from a non-archival cloud storage class.
- C. It's not possible to do it directly on Snowflake



POP QUIZ:

Mock Exam Questions



What are the steps to load data from an archival cloud storage class (for example, Amazon S3 Glacier or Microsoft Azure Archive Storage) into Snowflake?

- A. Same steps as loading data from a non-archival cloud storage class.
- B. You should create an ARCHIVAL stage; then, you can perform the same steps as loading data from a non-archival cloud storage class.
- C. It's not possible to do it directly on Snowflake

POP QUIZ:

Mock Exam Questions



After creating the table mytable, we execute the following commands: `CREATE STREAM MYSTREAM ON TABLE MYTABLE;`

`INSERT INTO MYTABLE VALUES (15);`

What will be the output after executing the following command?

`SELECT SYSTEM$STREAM_HAS_DATA('MYSTREAM');`

- A. It will return "True".
- B. It will return "False".
- C. It will return "15".
- D. It will return "Null".

POP QUIZ:

Mock Exam Questions



After creating the table mytable, we execute the following commands: `CREATE STREAM MYSTREAM ON TABLE MYTABLE;`

`INSERT INTO MYTABLE VALUES (15);`

What will be the output after executing the following command?

`SELECT SYSTEM$STREAM_HAS_DATA('MYSTREAM');`

- A. It will return "True".
- B. It will return "False".
- C. It will return "15".
- D. It will return "Null".

POP QUIZ:

Mock Exam Questions



Does an external function's identifier need to be unique for the schema in which the function is created?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



Does an external function's identifier need to be unique for the schema in which the function is created?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



Does an external function's identifier need to be unique for the schema in which the function is created?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



Which of the following JOIN statements are supported by Snowflake?

- A. Inner join
- B. Outer join
- C. Cross join
- D. Natural join
- E. All of the above



POP QUIZ:

Mock Exam Questions



Which of the following JOIN statements are supported by Snowflake?

- A. Inner join
- B. Outer join
- C. Cross join
- D. Natural join
- E. All of the above



POP QUIZ:

Mock Exam Questions



After creating a database and a schema using the following commands:

```
CREATE OR REPLACE DATABASE MY_DB  
DATA_RETENTION_TIME_IN_DAYS=30;
```

```
CREATE OR REPLACE SCHEMA S1  
DATA_RETENTION_TIME_IN_DAYS=50;
```

How Long will we be able to access the data from the schema using the Time Travel functionality if we drop the database?

- A. The default time on the account
- B. 30 days
- C. 50 days
- D. 90 days



POP QUIZ:

Mock Exam Questions



After creating a database and a schema using the following commands:

```
CREATE OR REPLACE DATABASE MY_DB  
DATA_RETENTION_TIME_IN_DAYS=30;
```

```
CREATE OR REPLACE SCHEMA S1  
DATA_RETENTION_TIME_IN_DAYS=50;
```

How Long will we be able to access the data from the schema using the Time Travel functionality if we drop the database?

- A. The default time on the account
- B. 30 days
- C. 50 days
- D. 90 days



POP QUIZ:

Mock Exam Questions



If you want your Data warehouse available with no lag time, is it a good idea to consider disabling the auto-suspend functionality?

- A. True
- B. False



POP QUIZ:

Mock Exam Questions



If you want your Data warehouse available with no lag time, is it a good idea to consider disabling the auto-suspend functionality?

- A. True
- B. False

Explanation: You might consider disabling this functionality in two cases; if you have a heavy, steady workload; and if you require the warehouse always to be available with no delay time. However, disabling this might incur more compute costs, so you should consider the pros and cons.



POP QUIZ:

Mock Exam Questions



Our company wants to execute a complicated ETL process inside Snowflake every day on the new data that is coming. What is the best way to automate it?

- A. Create an external script that runs every day at the same hour using AWS Lambda.
- B. Create a Snowflake task.
- C. Create a docker image and use it as an external function



POP QUIZ:

Mock Exam Questions



Our company wants to execute a complicated ETL process inside Snowflake every day on the new data that is coming. What is the best way to automate it?

- A. Create an external script that runs every day at the same hour using AWS Lambda.
- B. **Create a Snowflake task.**
- C. Create a docker image and use it as an external function

Explanation: The code runs inside Snowflake; it's as easy as creating a task to run it daily at a specific hour.



POP QUIZ:

Mock Exam Questions



Which is generally the slowest option when using the COPY INTO command?

- A. By path for internal stages and prefix for external
- B. Specifying a list of files to load
- C. Using pattern matching



POP QUIZ:

Mock Exam Questions



Which is generally the slowest option when using the COPY INTO command?

- A. By path for internal stages and prefix for external
- B. Specifying a list of files to load
- C. Using pattern matching

Explanation: These are the three options the COPY INTO command supports for loading data. Specifying a list of files to load is the fastest one, and using pattern matching is usually the slowest.

